



**DE MONTFORT
UNIVERSITY
LEICESTER**

Interceptor AI

AI-POWERED SOCIAL ENGINEERING ATTACKS SIMULATION

Aman Shabbir Patel
P2785848 | BS CYBERSECURITY

Acknowledgements:

I would like to express my sincere gratitude to my supervisor, **Mrs. Ismini Vasileiou** for their guidance, feedback, and support throughout the duration of this project. Their advice during both deliverable phases helped shape the direction of INTERCEPTOR and kept the work grounded in academic rigour.

I would also like to thank my family for their patience and encouragement during the long hours this project demanded. Their support made the process more manageable.

Finally, I acknowledge the open-source communities behind the tools and libraries that made this project possible - the developers of FastAPI, Next.js, Mongoose, sentence-transformers, and the Ollama project - whose freely available work provided the technical foundation on which INTERCEPTOR was built.

Abstract:

Social engineering attacks represent one of the most persistent and damaging threats in modern cybersecurity, exploiting human psychology rather than technical vulnerabilities to compromise individuals and organizations. Despite widespread investment in security awareness training, existing approaches remain largely static and fail to replicate the adaptive, conversational pressure of a real attack. This project presents INTERCEPTOR, an AI-powered social engineering awareness training platform that addresses this gap through realistic, real-time simulation.

INTERCEPTOR places users in live text-based conversations with an AI attacker whose behavior is driven by a Retrieval-Augmented Generation pipeline, a vector knowledge base of real social engineering tactics, and a large language model. Attack messages are personalized to the user's job role and assessed vulnerability profile and adapt dynamically based on how the user responds. Five attack scenarios are supported - phishing, business email compromise, MFA fatigue, and pretexting - each available across three difficulty levels. Upon session completion, users receive a Security Awareness Score, a letter grade, and a personalized debrief identifying which psychological triggers were exploited and providing targeted countermeasures.

The system is built across four independently running services - a Next.js frontend, a Next.js admin panel, an Express REST API, and a Python Fast API AI service - with MongoDB for persistent storage and Pinecone as the vector database. A comprehensive test suite of 204 automated tests across units, integration, system, black box, and non-functional categories validates the platform, achieving a 97% pass rate. The report documents the complete development of INTERCEPTOR from requirements and system design through implementation, testing, and critical evaluation, including honest reflection on design deviations, technical limitations, and future development directions.

Table of Contents:

Acknowledgements:.....	1
Abstract:	2
Introduction:.....	8
Aims and Objectives:.....	9
Problem Statement:.....	9
Aims:	9
Objectives:.....	9
System Architecture:	10
UML Use Case Diagram:.....	11
System Architecture Diagram:	13
ER Diagram (Database Schema):	15
Class Diagram:	17
Sequence Diagrams:.....	19
Authentication Sequence Diagram:	19
Admin KPI Analytics Sequence Diagram:.....	20
AI Message Processing - Flow Chart Diagram:	21
Functional Requirements Specification:	24
1. Authentication and User Management	24
FR 1: User Registration.....	24
FR 2: User Login	24
FR 3: Role-Based Access Control	25
FR 4: Password Reset	25
2. Assessment and Profiling.....	25
FR 5: Personality Assessment.....	25
FR 6: Technical Security Assessment	26
FR 7: Target Profile Generation.....	26

3. Simulation Session Management	26
FR 8: Scenario Selection	26
FR 9: Real-Time Chat Interface	27
FR 10: Persona Introduction and Consent	27
FR 11: Adaptive Tactic Retrieval (RAG).....	27
FR 12: Suspicion Detection and Pivot.....	28
FR 13: Session Termination and Outcome Recording	28
4. AI Orchestration	28
FR 14: Lang Chain Orchestration Loop	28
FR 15: Llama-Based Message Generation	29
FR 16: KPI Computation Engine	29
FR 17: Comprehensive Session Logging	29
FR 18: Admin Dashboard	30
Non-Functional Requirements (NFRs):	30
NFR 1: Performance - Page Load Time.....	30
NFR 2: Performance - LLM Response	30
NFR 3: Availability	31
NFR 4: Scalability	31
NFR 5: Security - Authentication and Authorization	31
NFR 6: Security - Data Encryption	31
NFR 7: Usability - Responsive Design	32
NFR 8: Usability - Accessibility	32
NFR 9: Reliability - Fault Tolerance	32
NFR 10: Maintainability	33
NFR 11: Compliance - GDPR and Data Privacy	33
NFR 12: Monitoring and Logging	33
Development Lifecycle:	34
Methodology:	34
Development Stages:.....	34

Stage 1 - Requirements and Design	34
Stage 2 - Core Infrastructure	35
Stage 3 - AI Service and RAG Pipeline.....	35
Stage 4 - Simulation Engine	35
Stage 5 - Supporting Features.....	36
Stage 6 - Testing, Refinement, and Evaluation.....	36
Summary of Verification and Validation Approach:	36
Implementation:	37
Overview.....	37
1. Frontend - User Interface	38
2. Express Backend - API Layer	46
Middleware Stack:.....	46
Route Architecture:	47
Authentication Middleware:	47
3. Authentication System	47
Registration and Email Verification:	47
Login and JWT Issuance:	48
Two-Factor Authentication:.....	48
Password Reset:.....	48
4. Simulation Engine	48
Phase 1 - Starting a Session:	48
Phase 2 - Turn-by-Turn Conversation:.....	49
Phase 3 - Session End and Scoring:	49
5. RAG Pipeline	50
Knowledge Base:.....	50
Embedding and Retrieval:	51
Reranking:	51
6. Learning Modules and Phishing Lab.....	51
7. Scoring and Debrief.....	53

8. Admin Dashboard	54
9. Landing Page Chatbot	59
Testing:.....	61
Overview:.....	61
1. Unit Tests.....	62
1.1 Authentication — TC-01 to TC-05	62
1.2 KPI Computation and Message Validation - TC-06, TC-12	63
1.3 Evaluator and Session Manager — Python	65
2. Integration Tests	66
3. System / End-to-End Tests	68
4. Black Box Functional Tests	69
5. Non-Functional Tests	71
6. AI Service Integration Tests - Python	73
Test Execution Output	75
Node.js - Jest (6 suites)	75
Python - Unit Tests.....	75
Python - Integration Tests	75
Critical Evaluation:	76
1. Deviations from Initial Design.....	76
1.1 LangChain Dropped - Custom Pipeline Built Instead.....	76
1.2 Gemma Replaced with Gemma 4B and OpenRouter	77
1.3 Next.js Server Actions Replaced with REST API Calls	77
1.4 MongoDB Schema Evolved from Initial ER Diagram	78
2. What Worked Well.....	78
2.1 The RAG Pipeline.....	78
2.2 SSE Streaming Architecture:	78
2.3 Authentication System	79
2.4 Modular Four-Service Architecture	79
3. Limitations and What Did Not Work as Planned	79

3.1 OpenRouter Free-Tier Rate Limiting	79
3.2 In-Memory AI Sessions	79
3.3 Vishing and Deepfake Scenarios Not Implemented	80
3.4 Pinecone Free Tier Constraints	80
4. Tool Evaluation.....	80
5. Future Work	81
Conclusion:	82
References:	83

Introduction:

Cybersecurity threats continue to evolve at a rapid pace, with social engineering remaining one of the most effective and damaging attack vectors faced by organizations today. Unlike technical exploits that target system vulnerabilities, social engineering attacks target human psychology - exploiting trust, authority, urgency, and fear to manipulate individuals into disclosing sensitive information or performing actions that compromise security. High-profile incidents such as the MGM Resorts vishing attack of 2023, which cost the company approximately \$100 million, and the Uber MFA fatigue breach of 2022, demonstrate that even organizations with mature technical defenses remain vulnerable when their people are unprepared.

Traditional security awareness training has struggled to keep pace with the sophistication of modern attacks. Static videos, annual compliance modules, and generic phishing simulations fail to replicate the adaptive, conversational nature of real social engineering campaigns. They do not expose individuals to the psychological pressure of a live interaction, and they rarely provide meaningful, personalized feedback on where a person's specific vulnerabilities lie.

INTERCEPTOR was developed to address this gap. It is an AI-powered social engineering awareness training platform that places users in realistic, adaptive simulation scenarios driven by a large language model. Users engage in live conversations with an AI attacker whose behavior is shaped by retrieved social engineering tactics, the chosen scenario type, and the user's own assessed vulnerability profile. After each session, the system computes a personalized Security Awareness Score and provides a structured debrief identifying which psychological triggers were exploited and how the user's responses are compared to best practice.

This report documents the complete development of INTERCEPTOR from specification through to implementation and evaluation. It covers the system architecture underpinning the platform, the functional and non-functional requirements that guided development, the implementation of each major component - including the simulation engine, RAG pipeline, scoring system, and administrative dashboard - and the testing carried out to validate the system. A critical evaluation concludes the report, reflecting on the development process, the tools and technologies used, and directions for future work.

Aims and Objectives:

Problem Statement:

Cybersecurity threats driven by social engineering continue to rise in frequency and sophistication. Despite significant investment in technical defences, human vulnerability remains the primary attack surface exploited by adversaries. Existing security awareness training programmes - static videos, annual compliance modules, and generic phishing simulations - fail to replicate the adaptive, conversational pressure of a real social engineering attack and provide little personalised feedback on individual vulnerabilities. There is a clear need for a training platform that places users inside realistic, AI-driven attack scenarios and delivers specific, evidence-based feedback tied to their actual behaviour.

Aims:

The aim of this project is to design, develop, and evaluate INTERCEPTOR - an AI-powered social engineering awareness training platform that simulates realistic attack scenarios in real time, assesses user behavior, and delivers personalized debriefs to improve security awareness.

Objectives:

- Conduct a literature review of social engineering attack types, psychological manipulation techniques, AI-driven phishing, and existing security awareness training approaches to establish the academic foundation for the project.
- Design a complete system architecture covering the frontend, backend API, AI service, and database layers, with clear separation of responsibilities between each component.
- Implement a functional authentication system supporting email/password login, email verification, two-factor authentication, and role-based access control for trainee and administrator accounts.

- Build a simulation engine that places users in real-time text-based conversations with an AI attacker, where attacker messages are grounded in retrieved social engineering tactics and adapt based on user responses.
- Implement a Retrieval-Augmented Generation pipeline using a vector database, sentence embedding model, and cross-encoder reranker to retrieve contextually relevant attack tactics and debrief content at runtime.
- Design and implement a scoring and debrief system that evaluates user responses turn-by-turn, computes an overall Security Awareness Score, assigns a letter grade, and produces personalized feedback identifying which psychological triggers were exploited.
- Develop structured learning modules covering key social engineering topics, a phishing lab for identifying fake emails and login pages, and an administrative dashboard for platform management and analytics.
- Produce a comprehensive test suite covering unit, integration, system, black box, and non-functional tests to validate that all functional and non-functional requirements are met.
- Critically evaluate the completed system against the original specification, reflecting on design deviations, technical limitations, tool choices, and directions for future development.

System Architecture:

The system follows a layered architecture that separates user interaction, application logic, artificial intelligence orchestration, and data storage components. This approach improves scalability, maintainability, and security.

The frontend interface is implemented using Next.js, which provides both client-side rendering and server-side functionality through server actions. These server actions function as API endpoints responsible for handling authentication, session management, and communication with backend services.

The orchestration layer is implemented using Lang Chain. This part coordinates the interaction between the language model and the retrieval system. When a user message is received, the orchestrator analyses the content and decides which social engineering tactic should be retrieved.

Relevant tactics are retrieved from the vector database using Pinecone. Each tactic is stored as an embedding that allows efficient similarity search. The retrieved tactics are then included in the prompt sent to the language model.

The Llama 3.1 language model generates the conversational responses. This model produces contextually proper attacker messages that simulate realistic social engineering behaviour.

All session data, user information, and analytics are stored in MongoDB. This database also records user interactions and KPI events that are later analysed through the administrative dashboard.

UML Use Case Diagram:

The use case diagram illustrates the primary interactions between system users and the AI-driven social engineering training platform. Two main actors interact with the system: the **Employee**, who participates in simulated training sessions, and the **Security Officer (Administrator)**, who monitors system performance and user behavior.

This diagram represents a **bridge between a user (Employee) and an AI (Attacker)** to test security awareness.

1. The Core Interaction (The "Brain")

The most important part of the diagram is the **Interact with AI Attacker** section.

- It shows that every time an employee speaks, the system must **Retrieve Social Engineering Tactics** (the "strategy") and **Generate AI Response** (the "speech").
- Think of this as the AI "thinking" before it "speaks" to try and trick the employee.

2. The Feedback Loop

The diagram shows a clear split between **Action** and **Analysis**:

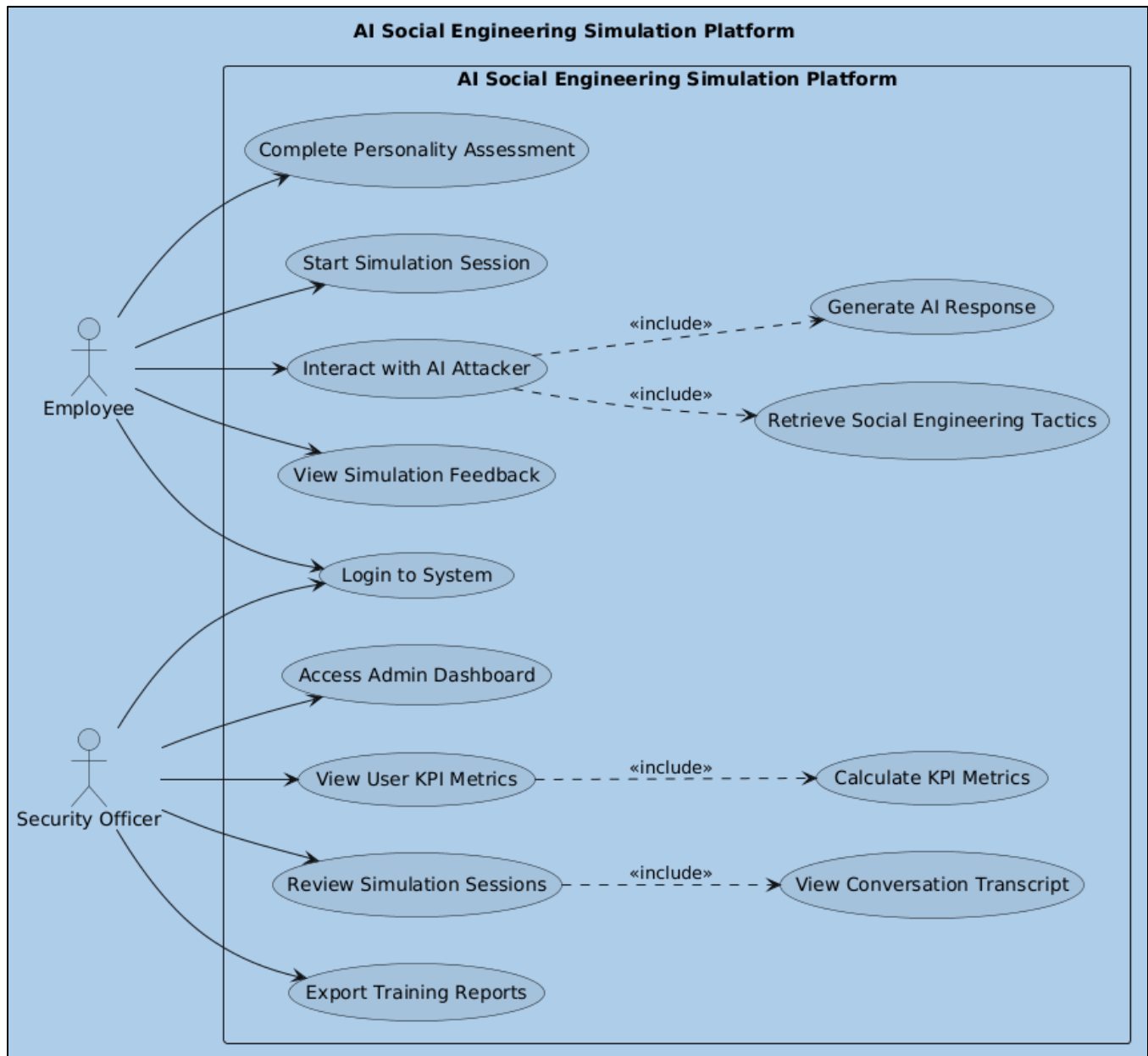
- **Employee Side:** They do the "work" (Assessments, Simulations).
- **Security Officer Side:** They do the "oversight." They do not just see a "Pass/Fail" grade; they can drill down into **Conversation Transcripts** to see exactly where an employee was vulnerable.

3. The Functional Flow (Simplified)

If you follow the arrows, the process looks like this:

1. **Preparation:** Employee takes a personality test.

2. **Execution:** Employee chats with the AI (which uses LLM and RAG tactics).
3. **Reporting:** The system calculates KPIs (Key Performance Indicators).
4. **Review:** The Security Officer exports a report for the company.



System Architecture Diagram:

This diagram is a **System Architecture Diagram** that shows how the platform is actually built. It explains how data flows from the user's screen into the AI "brain" and back.

Here is the high-level breakdown of each layer:

1. User Interface (The Frontend)

- **Employee Chat UI:** Where the trainee talks to the AI.
- **Admin Dashboard:** Where the security officer sees the analytics.
- Both communicate via **API Endpoints (Server Actions)** in a **Next.js** environment.

2. Orchestration Layer (The "Brain")

- **LangChain.js Orchestrator:** This is the most important part. It manages "Agent Loops." Instead of just sending a message to an AI, it decides *how* to handle the message—checking history, pulling tactics, and then deciding what to say next.

3. Data & AI Layer (The "Memory")

- **Pinecone (Vector DB):** This stores the social engineering tactics. It allows the AI to "search" for a specific strategy (like phishing) based on the conversation context (**RAG - Retrieval Augmented Generation**).
- **Gemma (Inference):** This is the actual Large Language Model (LLM) that generates the human-like text responses.
- **MongoDB (Primary DB):** This stores standard data like user accounts, profiles, and simulation logs.

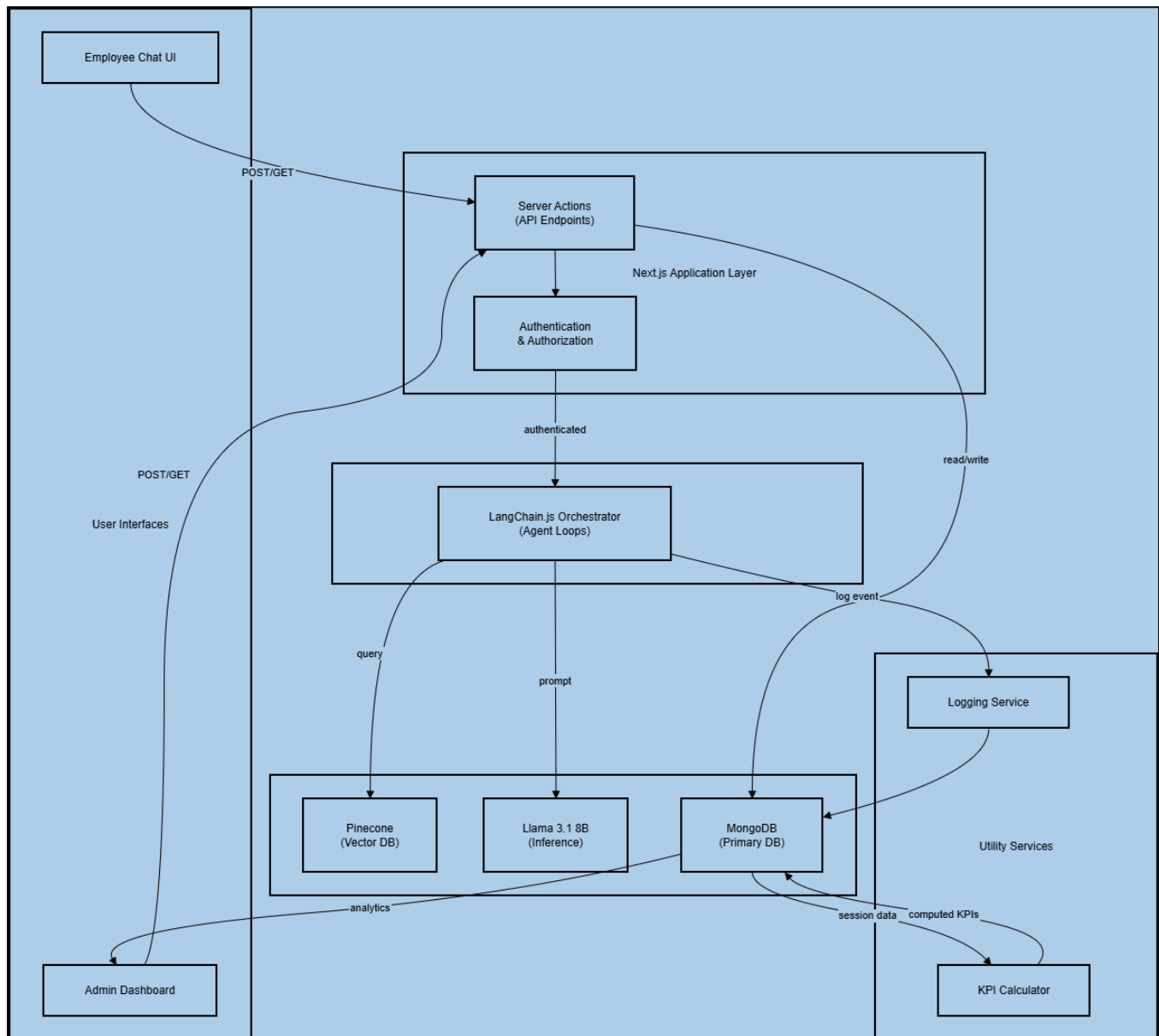
4. Utility Services (The "Analytics")

- **Logging Service:** Records every event for security and review.
- **KPI Calculator:** Takes the "Session Data" from MongoDB and calculates the scores (Key Performance Indicators) that the admin sees on their dashboard.

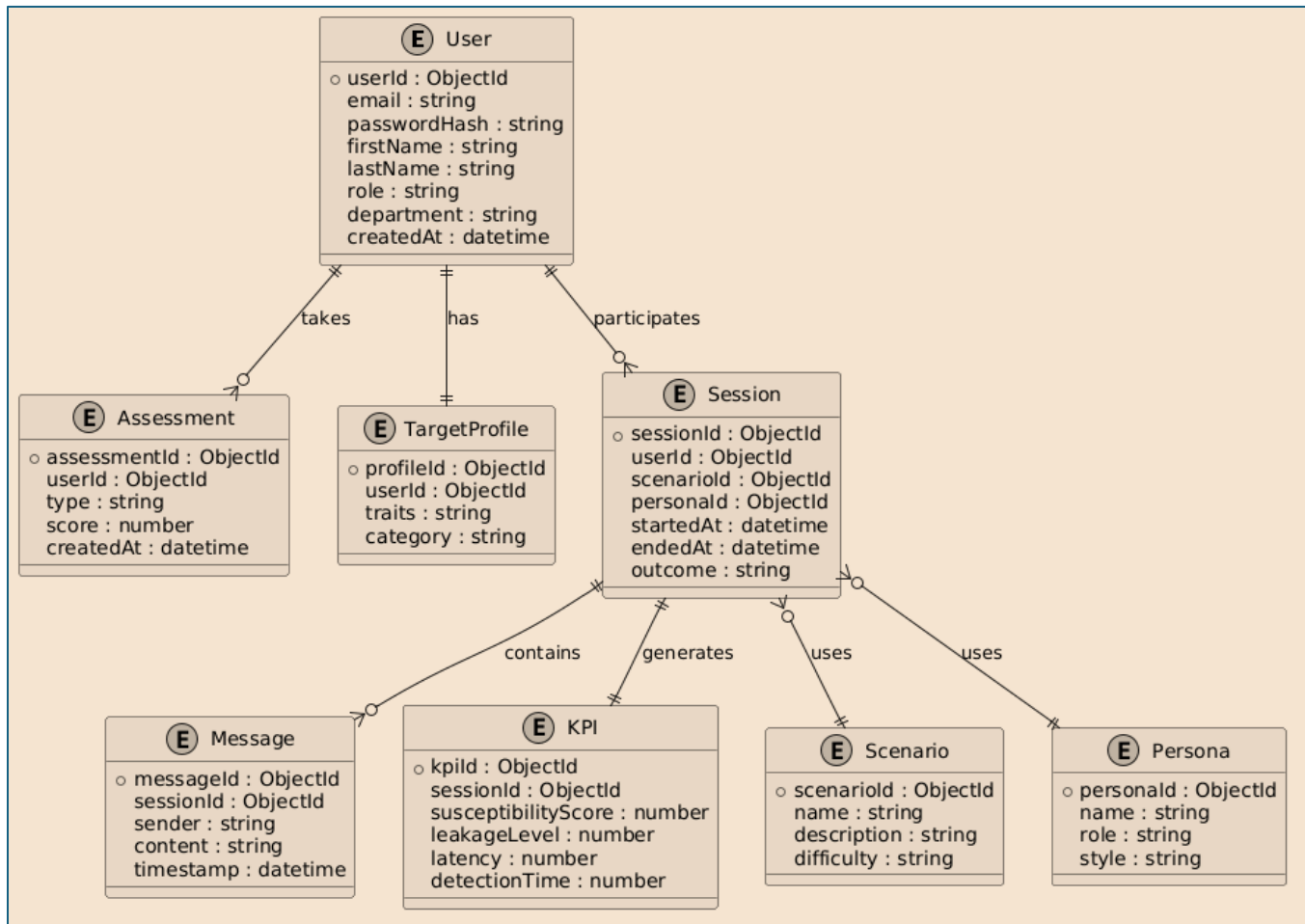
Simplified Flow of a Message:

1. **Employee** sends a message in the **Chat UI**.
2. **Next.js** sends it to the **Lang Chain Orchestrator**.

3. **Lang Chain** queries **Pinecone** for a tactic and sends a prompt to **Llama 3.1**.
4. The response is logged in **MongoDB** and sent back to the **Employee**.
5. The **KPI Calculator** later analyses that log to update the **Admin Dashboard**.



ER Diagram (Database Schema):



ER Diagram Description

The ER diagram represents the database structure of the AI-driven social engineering training platform. It models the key entities and their relationships required to manage users, simulations, and analytics.

Core Entities

1. User

Represents both trainees and administrators.

Stores authentication and profile information such as email, role, and department.

2. Assessment

Stores results of personality and technical assessments.

Each user can have multiple assessments over time.

Relationship:

User → Assessment = One-to-Many

3. Target Profile

Represents the vulnerability profile generated from assessments.

Each user has one active target profile.

Relationship:

User → Target Profile = One-to-One

4. Session

Represents a simulation session between a trainee and the AI attacker.

Stores scenario, persona, timestamps, and outcome.

Relationship:

User → Session = One-to-Many

5. Message

Stores chat messages exchanged during a session.

Includes both user and AI-generated messages.

Relationship:

Session → Message = One-to-Many

6. KPI

Stores computed performance metrics after a session.
Includes susceptibility score, leakage level, latency, and detection time.

Relationship:

Session → KPI = One-to-One

7. Scenario

Defines the type of social engineering attack (e.g., CEO fraud, phishing).

8. Persona

Defines the attacker's behaviour and communication style.

Relationships Summary

- A **User** can have multiple **Sessions**
- A **Session** uses one **Scenario** and one **Persona**
- A **Session** contains multiple **Messages**
- A **Session** generates one **KPI record**
- A **User** has one **Target Profile**

Class Diagram:

Class Diagram Description

The class diagram represents the internal structure of the system using object-oriented design principles. It defines the main classes, their attributes, methods, and relationships.

Core Classes

1. User

Represents system users (trainee or admin).

Handles authentication and account management through methods like register (), login (), and reset Password().

2. Assessment

Stores results of personality and technical tests.
Includes logic to calculate scores.

3. Target Profile

Represents the behavioural profile derived from assessments.
Used to personalize AI-driven simulations.

4. Session

Represents a simulation session.
Manages lifecycle using start Session() and end Session().

5. Message

Represents individual chat messages exchanged between user and AI.
Each session contains multiple messages.

6. KPI

Stores performance metrics such as susceptibility score and leakage level.
Generated after session completion using compute().

7. Scenario

Defines the attack scenario (e.g., phishing, CEO fraud).

8. Persona

Defines attacker characteristics such as tone, role, and communication style.

9. AI Engine

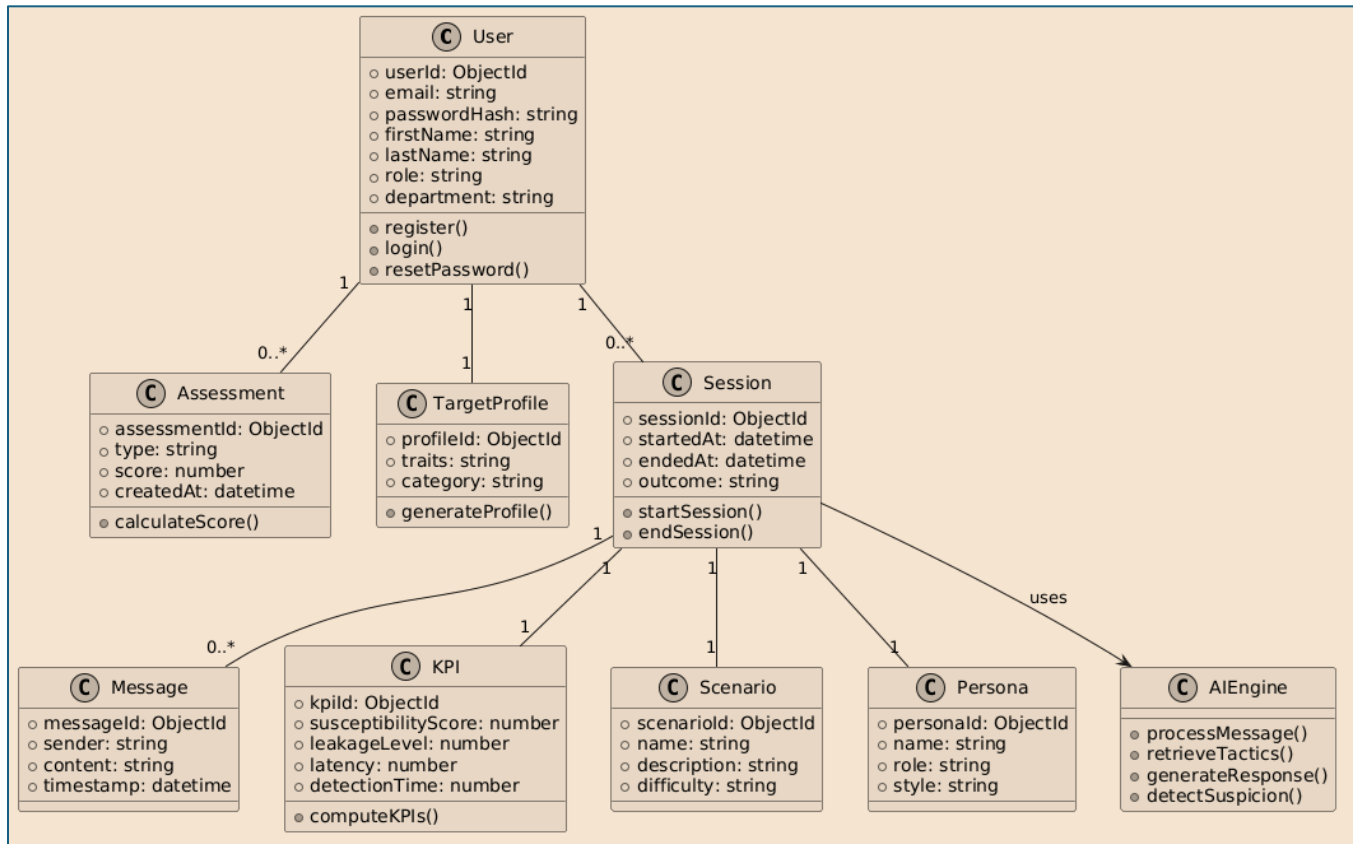
Represents the AI orchestration logic.
Handles:

- Message processing
- Tactic retrieval (RAG)
- Response generation (LLM)
- Suspicion detection

Relationships

- A **User** can have multiple **Sessions** and **Assessments**
- A **Session** contains multiple **Messages**

- A **Session** is linked to one **Scenario** and one **Persona**
- A **Session** generates one **KPI record**
- The **AI Engine** is used by the Session to process interactions

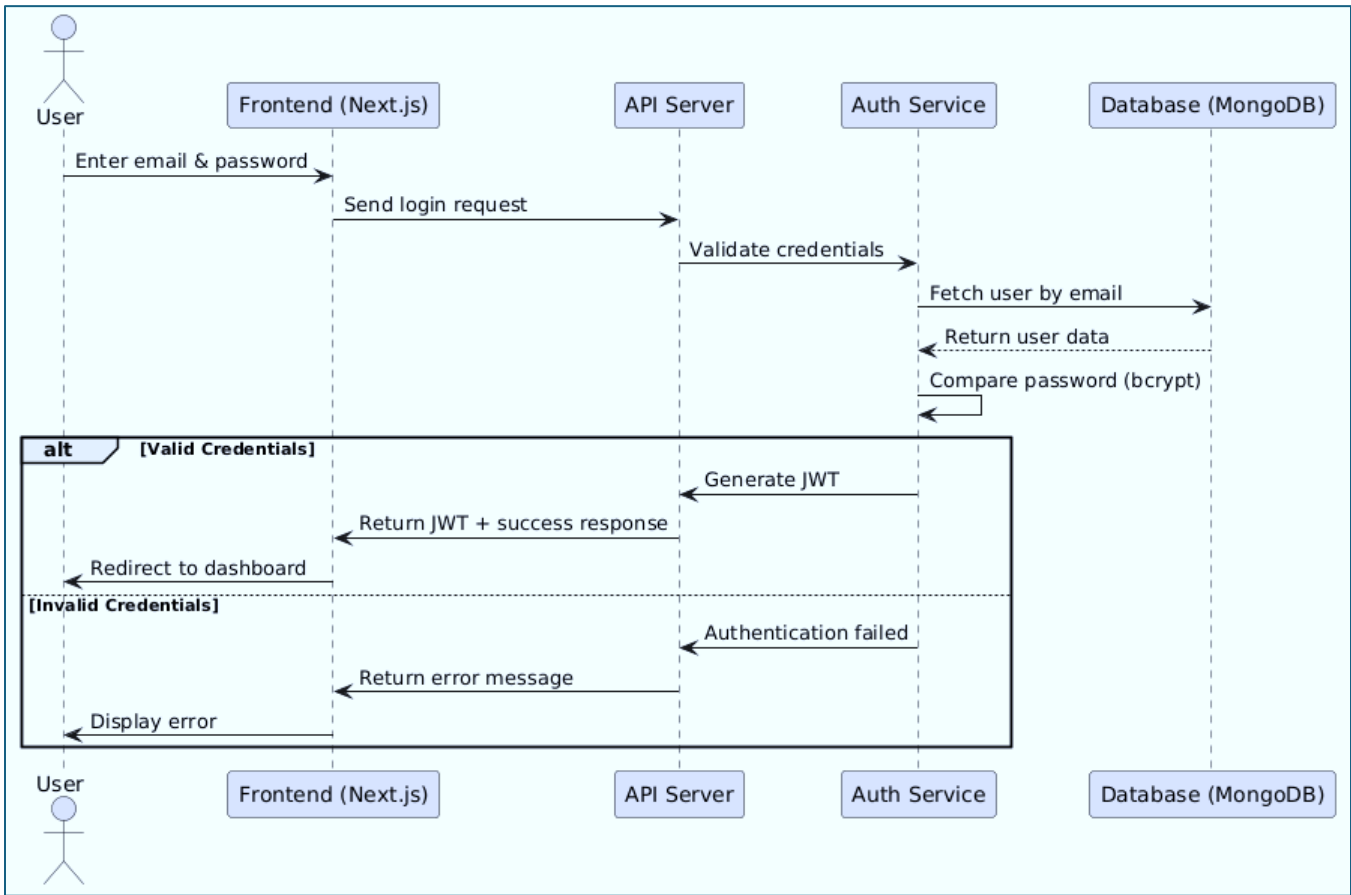


Sequence Diagrams:

Authentication Sequence Diagram:

The authentication sequence diagram represents the step-by-step interaction between the user and system components during the login process. When a user submits their email and password through the frontend interface, the request is sent to the backend server (API layer). The system then validates the credentials by comparing the provided password with the stored hashed password in the database. If the credentials are valid, a JSON Web Token (JWT) is generated and returned to the client, allowing the user to access protected resources such as the dashboard.

If the credentials are invalid, the system returns an error response and logs the failed attempt for security monitoring. Additional security measures, such as account lockout after multiple failed attempts, are enforced at this stage. Overall, the process ensures that only authenticated users can access the platform while maintaining secure session management and protecting against unauthorized access.

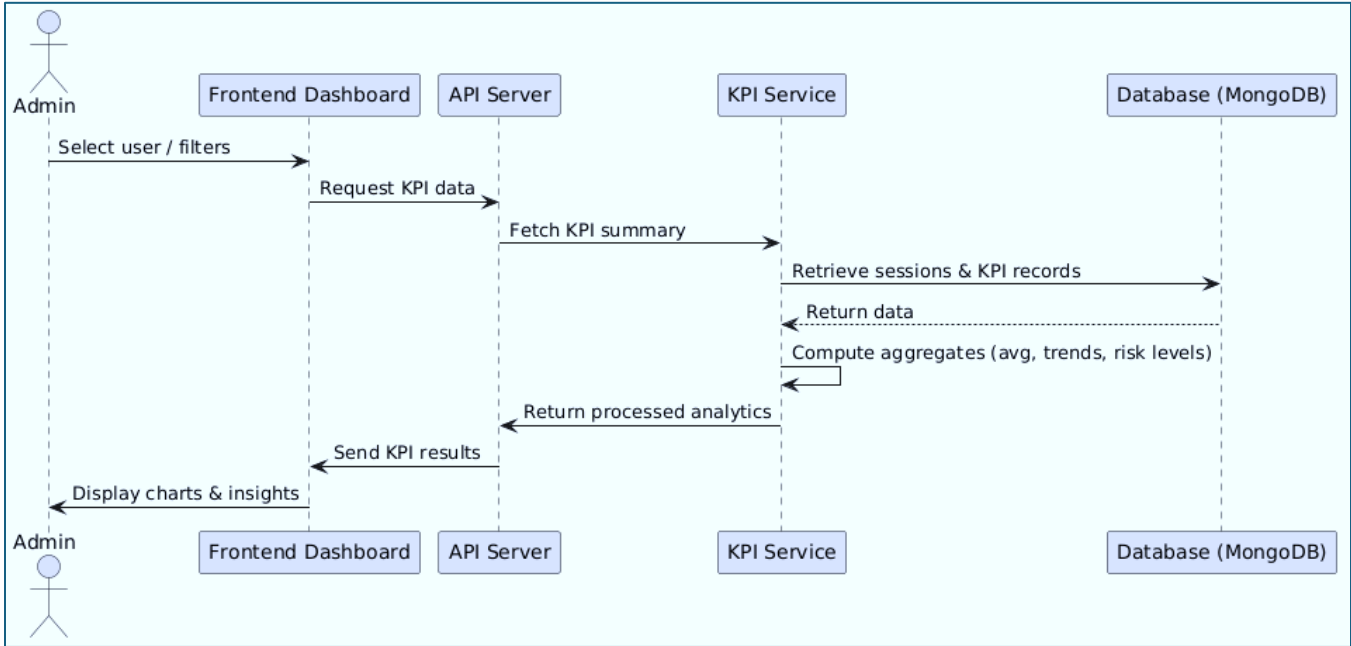


Admin KPI Analytics Sequence Diagram:

The Admin KPI Analytics sequence diagram illustrates how the system retrieves and processes performance data for display on the administrative dashboard. When an administrator selects a user or applies filters, the frontend sends a request to the backend API. The API forwards this request to the KPI service, which retrieves relevant session data and previously computed KPI records from the database.

The KPI service then processes this data to compute aggregated metrics such as average susceptibility scores, trends over time, and risk classifications. These processed analytics are

returned to the API and then sent to the frontend, where they are visualized as charts and insights for the administrator. This process enables efficient monitoring of user performance and supports data-driven security training decisions.

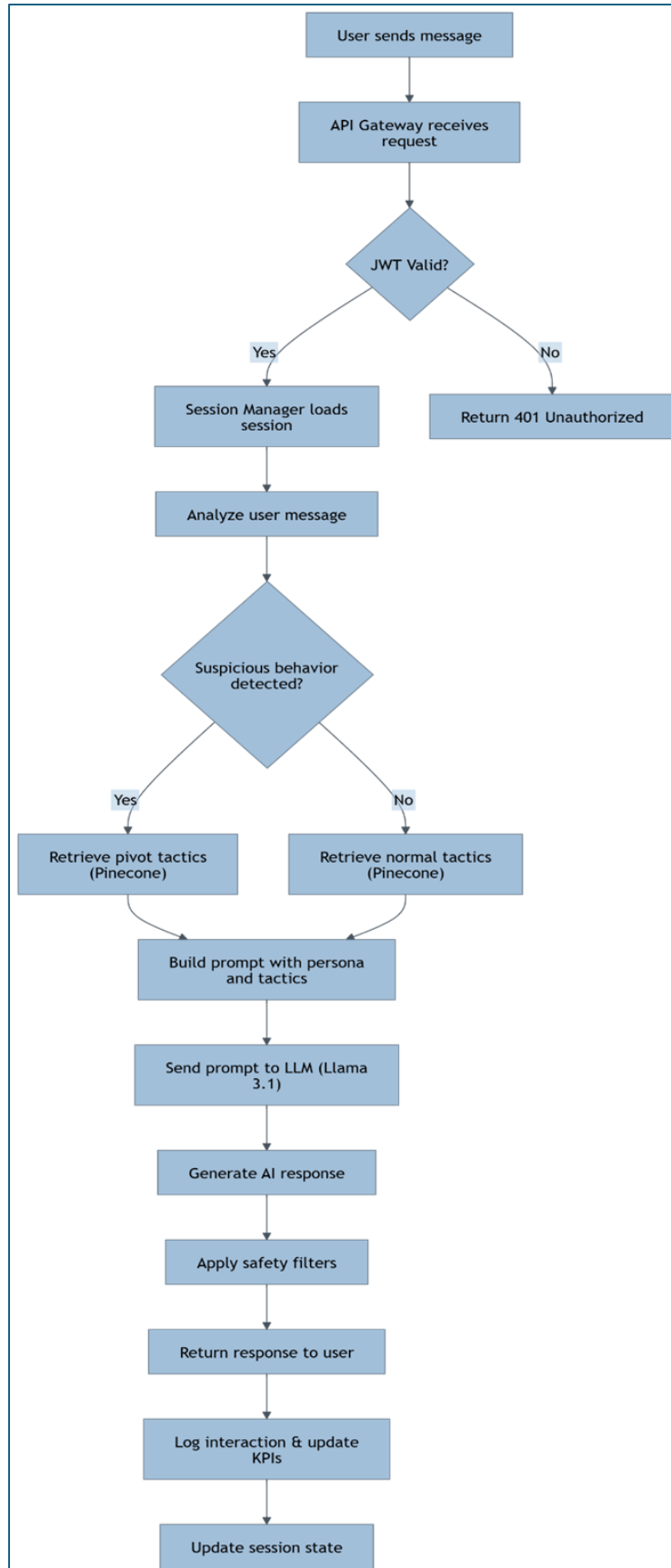


AI Message Processing - Flow Chart Diagram:

The AI Message Processing flowchart illustrates the internal workflow followed when a user sends a message during a simulation session. The process begins with the API validating the user’s authentication token and retrieving the active session. The system then analyses the user’s message to determine intent and detect any signs of suspicion. Based on this analysis, the system retrieves appropriate social engineering tactics from the vector database, either continuing the attack strategy or pivoting in response to user scepticism.

The retrieved tactics, along with persona guidelines and conversation history, are used to construct a prompt for the language model. The LLM generates a context-aware response, which is then passed through safety filters before being returned to the user. Finally, the system logs the

interaction, updates session data, and records KPI-related metrics, enabling continuous tracking of user behaviour and performance throughout the simulation.



Functional Requirements Specification:

1. Authentication and User Management

FR 1: User Registration

Priority: Critical

Actor: New User

The system shall allow new users to create an account by providing email, password, first name, last name, and department.

Acceptance Criteria:

- Validate email format (RFC 5322).
- Enforce password strength: ≥8 characters, 1 uppercase, 1 lowercase, 1 number, 1 special character.
- Prevent duplicate emails (case-insensitive).
- Store verified accounts in MongoDB.
- Send confirmation email with verification link.

FR 2: User Login

Priority: Critical

Actor: Registered User

The system shall authenticate users using email and password.

Acceptance Criteria:

- Validate credentials against stored bcrypt hashes.
- Issue JWT valid for 8 hours upon successful login.
- Lock account for 30 minutes after 5 consecutive failed attempts.
- Provide error messages for invalid login attempts.

FR 3: Role-Based Access Control

Priority: Critical

Actor: All Users

The system shall enforce role-based access control for trainee and admin functionality.

Acceptance Criteria:

- Trainees cannot access admin routes (/dashboard, /scenarios, /personas).
- Admins cannot access trainee simulation routes.
- Log and reject unauthorized attempts with 403 responses.

FR 4: Password Reset

Priority: High

Actor: Registered User

The system shall offer a self-service password reset mechanism.

Acceptance Criteria:

- Generate 24-hour time-limited reset tokens.
- Send reset link via email.
- Allow user to enter a new password meeting strength requirement.
- Invalidate old reset tokens after use.

2. Assessment and Profiling

FR 5: Personality Assessment

Priority: Critical

Actor: Trainee

The system shall administer a 50-question Big Five (OCEAN) personality assessment.

Acceptance Criteria:

- Score each dimension 0–100.
- Require all questions to be answered before going ahead.
- Store results in MongoDB with timestamp.
- Provide trainee feedback and interpretation.

FR 6: Technical Security Assessment

Priority: High

Actor: Trainee

The system shall administer a 20-question multiple-choice security awareness quiz.

Acceptance Criteria:

- Score each answer; total score 0–20.
- Display results at once with explanations for incorrect answers.
- Track historical attempts and recommend training modules for weak areas.

FR 7: Target Profile Generation

Priority: Critical

Actor: System

The system shall generate a Target Profile based on assessment results.

Acceptance Criteria:

- Map Big Five and technical scores to profile categories.
- Store profiles in MongoDB for session personalization.
- Support at least 8 distinct profile types.

3. Simulation Session Management

FR 8: Scenario Selection

Priority: Critical

Actor: Trainee

The system shall allow trainees to select a scenario and attacker persona to begin a session.

Acceptance Criteria:

- Display available scenarios and personas.
- Initialize a new session document in MongoDB with timestamps.
- Load trainee Target Profile for adaptive simulation.

FR 9: Real-Time Chat Interface

Priority: Critical

Actor: Trainee

The system shall provide a chat interface for trainee-attacker interaction.

Acceptance Criteria:

- Display conversation history.
- Trigger AI response within 2.5 seconds.
- Support message input with line breaks and emojis.
- Limit sessions to 50 turns; allow manual termination.

FR 10: Persona Introduction and Consent

Priority: High

Actor: Trainee

The system shall show the attacker persona and scenario goal before starting the session.

Acceptance Criteria:

- Trainee must explicitly consent to take part.
- Log consent or decline in KPI Events.

FR 11: Adaptive Tactic Retrieval (RAG)

Priority: Critical

Actor: System

The system shall retrieve persona-aligned social engineering tactics from Pinecone.

Acceptance Criteria:

- Use profile tags and persona filters for retrieval.
- Provide fallback to local tactic library if Pinecone unavailable.
- Display retrieved tactics in event log.

FR 12: Suspicion Detection and Pivot

Priority: High

Actor: System

The system shall detect trainee suspicion and adjust attacker tactics accordingly.

Acceptance Criteria:

- Detect suspicion via keyword/NLP confidence >0.7.
- Retrieve pivot tactics from Pinecone to address user concerns.
- Log pivot events for KPI analysis.

FR 13: Session Termination and Outcome Recording

Priority: High

Actor: Trainee or System

The system shall record session termination and classify outcome.

Acceptance Criteria:

- Record session end reasons: manual abort, max turns, detection, disclosure.
- Assign outcome: attack success, attack_partially_success, attack_failed, user_aborted.
- Trigger KPI computation.

4. AI Orchestration

FR 14: Lang Chain Orchestration Loop

Priority: Critical

Actor: System

The system shall implement a React-style loop: Retrieve → Generate → Evaluate → Log.

Acceptance Criteria:

- Complete each loop in <2.5 seconds.
- Maintain conversation state in MongoDB.
- Include evaluation for suspicion and leakage.

FR 15: Llama-Based Message Generation

Priority: Critical

Actor: System

The system shall generate attacker messages using Gemma.

Acceptance Criteria:

- Condition messages on persona, tactics, last 5 turns.
- Enforce content constraints (no real credentials, no external links).
- Limit messages to <500 characters.

FR 16: KPI Computation Engine

Priority: Critical

Actor: System

The system shall compute KPIs after session completion.

Acceptance Criteria:

- Compute Susceptibility Score (0–100), Leakage Level (0–4), Response Latency, Time to Detection.
- Store results in KPI Summaries.
- Provide feedback to trainee and admin.

FR 17: Comprehensive Session Logging

Priority: Critical

Actor: System

The system shall log all session interactions and KPI events.

Acceptance Criteria:

- Include timestamps, user messages, attacker responses, latency, suspicion indicators, tactic references.
- Encrypt logs at rest.
- Retain for 12 months.

FR 18: Admin Dashboard

Priority: High

Actor: Admin

The system shall provide a dashboard for KPI analytics and user management.

Acceptance Criteria:

- Display aggregated KPIs, trends, and high-risk cohorts.
- Allow filtering by department, persona, scenario.
- Support CSV/PDF export of analytics and reports.

Non-Functional Requirements (NFRs):

NFR 1: Performance - Page Load Time

Priority: High

The system shall ensure that web pages load fully within 2 seconds under normal load conditions.

Rationale: Fast load improves trainee engagement and reduces frustration.

Acceptance Criteria:

- First content full paint < 2 seconds.
- Chat interface ready to accept input at once after page load.

NFR 2: Performance - LLM Response

Priority: High

The system shall deliver AI-generated responses in the chat interface within 2.5 seconds (p95 latency).

Acceptance Criteria:

- Lang Chain orchestration loop completes < 2.5 seconds.
- Users do not experience noticeable lag in conversation flow.

NFR 3: Availability

Priority: High

The system shall keep at least 99% uptime during business hours (8 AM–6 PM UTC).

Acceptance Criteria:

- Planned maintenance $\leq$ 2 hours/month.
- Unplanned downtime $\leq$ 1 hour/month.
- Core chat and assessment functions stay operational during partial outages.

NFR 4: Scalability

Priority: Medium

The system shall support concurrent users and scale dynamically.

Acceptance Criteria:

- Minimum 50 simultaneous sessions in Phase 1.
- Load balancing across API servers.
- Auto-scaling MongoDB and LLM inference queueing.

NFR 5: Security - Authentication and Authorization

Priority: Critical

The system shall secure user access using role-based authentication and JWT tokens.

Acceptance Criteria:

- JWT tokens expire in 8 hours.
- Passwords stored as bcrypt hashes.
- Unauthorized access attempts logged and rejected with 403 responses.

NFR 6: Security - Data Encryption

Priority: Critical

The system shall encrypt sensitive data at rest and in transit.

Acceptance Criteria:

- MongoDB encryption-at-rest enabled.

- TLS 1.2+ for all HTTP communication.
- Logs having sensitive information are encrypted.

NFR 7: Usability - Responsive Design

Priority: High

The system shall support desktop, tablet, and mobile devices with consistent user experience.

Acceptance Criteria:

- Minimum touch target size: 44×44 pox on mobile.
- Landscape and portrait orientations supported.
- No horizontal scrolling on mobile devices.

NFR 8: Usability - Accessibility

Priority: Medium

The system shall follow WCAG 2.1 AA accessibility standards.

Acceptance Criteria:

- Screen reader support via semantic HTML and ARIA labels.
- Keyboard navigation for all functions.
- Colour contrast $\geq 4.5:1$ for text.

NFR 9: Reliability - Fault Tolerance

Priority: High

The system shall manage service interruptions gracefully without data loss.

Acceptance Criteria:

- If Pinecone retrieval fails, fallback to local tactic library.
- Chat session state restored after temporary connection drop.
- Automatic retries with exponential backoff for transient failures.

NFR 10: Maintainability

Priority: Medium

The system shall be modular and maintainable to support future enhancements.

Acceptance Criteria:

- Codebase follows clean architecture principles.
- APIs are well-documented and versioned.
- Logs and metrics allow troubleshooting without code changes.

NFR 11: Compliance - GDPR and Data Privacy

Priority: Critical

The system shall follow GDPR and organizational data privacy policies.

Acceptance Criteria:

- Explicit user consent obtained before registration.
- Users can export or cut their data.
- Data retention policies enforced (default 12 months).
- Audit logs append-only and timestamped.

NFR 12: Monitoring and Logging

Priority: High

The system shall provide monitoring for performance, errors, and security events.

Acceptance Criteria:

- Real-time alerts for critical errors or security events.
- Session logs include latency, suspicion indicators, and KPI events.
- Dashboard shows system health metrics (uptime, API response time, LLM queue length).

Development Lifecycle:

Methodology:

INTERCEPTOR was developed using an iterative and incremental methodology - a lightweight approach aligned with Agile principles suited to a solo development project with evolving technical requirements. Rather than defining the complete system upfront and building it in a single pass, the project was divided into a series of development iterations, each of which produced a working, testable increment of the system. At the end of each iteration, the increment was reviewed, tested, and refined before the next began.

This approach was chosen over a strict Waterfall model for two reasons. First, several technical decisions could not be made with certainty at the outset - the choice of language model, the viability of LangChain as an orchestration layer, and the structure of the RAG pipeline all required practical experimentation before they could be finalised. An iterative model accommodates this uncertainty by allowing decisions to be revisited between cycles. Second, building and testing each component before moving to the next reduced the risk of integration failures late in the project, where debugging becomes expensive.

Development Stages:

Stage 1 - Requirements and Design

The first stage established the project foundation. The literature review was conducted to understand the social engineering threat landscape, existing training approaches, and relevant AI techniques including RAG and LLM-based generation. Functional and non-functional requirements were specified, and system design artefacts were produced: the UML use case diagram, system architecture diagram, entity-relationship diagram, class diagram, and sequence diagrams. This stage also defined the five attack scenarios, the scoring model, and the database schema.

Validation at this stage: Requirements were reviewed against the project specification to ensure completeness. Design diagrams were cross-checked for consistency - for example, verifying that every entity in the ER diagram had a corresponding class in the class diagram, and that sequence diagrams matched the defined API endpoints.

Stage 2 - Core Infrastructure

The second stage built the foundational infrastructure that all other components depend on. This included the Express backend with its middleware stack, MongoDB connection, and route architecture; the Next.js frontend shell with routing and layout components; and the FastAPI AI service skeleton with health check and configuration loading. The authentication system - registration, email verification, login, JWT issuance, password reset, and two-factor authentication - was implemented and tested fully before any simulation logic was built.

Validation at this stage: Unit tests for authentication (TC-01 to TC-05) were written and executed alongside the implementation. Each individual function - password hashing, token generation, JWT verification, bcrypt comparison - was verified in isolation before being integrated into the route handlers. Integration tests confirmed end-to-end authentication flows.

Stage 3 - AI Service and RAG Pipeline

The third stage implemented the AI service. This was the most technically uncertain stage of the project and required the most iteration. LangChain was trialled as the orchestration layer and abandoned after encountering version conflicts and unpredictable agent behaviour. A custom pipeline was built directly using httpx, sentence-transformers, and Pinecone. The embedding model, Pinecone index, and cross-encoder reranker were configured and the knowledge base was seeded. The session manager, trigger detector, rule-based evaluator, and LLM evaluator were implemented individually and unit tested.

Validation at this stage: Python unit tests (58 tests) validated every component of the AI service in isolation - grade boundaries, rule-based evaluation patterns, score merging weights, session lifecycle, trigger detection, and reasoning stripping. The RAG pipeline was validated by inspecting retrieved chunks against expected scenario types and chunk types to confirm that metadata filtering was working correctly.

Stage 4 - Simulation Engine

The fourth stage integrated the AI service with the Express backend to produce the working simulation flow. The three-phase simulation lifecycle - start, respond, end - was implemented, with both blocking and streaming response variants. Server-Sent Events streaming was implemented end-to-end from Ollama through FastAPI through Express to the browser. The parallel evaluation thread was introduced at this stage to allow LLM evaluation to run concurrently with message streaming.

Validation at this stage: Integration tests (TC-07 to TC-13) validated cross-component data flow - that session documents were created correctly in MongoDB, that trigger data was persisted per turn, that evaluation classifications matched expected outcomes for known inputs, and that

completed sessions contained all required KPI fields. End-to-end tests simulated complete user workflows from registration through to session completion.

Stage 5 - Supporting Features

The fifth stage built the remaining user-facing features: learning modules with quiz scoring and progress tracking, the phishing lab, the user history and results screens, the admin dashboard with analytics, the landing page chatbot, and the notification system. The admin panel was built as a separate Next.js application connecting to the same Express API.

Validation at this stage: Black box functional tests (BB-01 to BB-08) validated system behaviour from the outside - role-based access control, input boundary handling, token security, and session lifecycle. Non-functional tests (NFR-01 to NFR-12) measured response times, concurrency behaviour, security header presence, and fault tolerance.

Stage 6 - Testing, Refinement, and Evaluation

The final stage focused on completing the full test suite, resolving issues identified during testing, and producing the project documentation. The 204-test suite was assembled, executed, and results recorded. System behaviour under edge cases - malformed inputs, expired tokens, rate-limited APIs, concurrent requests - was verified and any issues resolved. The critical evaluation was written based on direct experience with the system throughout development.

Validation at this stage: The complete test suite served as the final verification pass. All 120 JavaScript tests and 58 Python unit tests passed. AI service integration tests confirmed end-to-end pipeline behaviour against the live OpenRouter API.

Summary of Verification and Validation Approach:

Stage	V&V Approach
Requirements & Design	Requirements review, diagram consistency checks
Core Infrastructure	Unit tests for auth, integration tests for flows
AI Service & RAG	Python unit tests (58), manual inspection of retrieved chunks
Simulation Engine	Integration and E2E tests, streaming behaviour verification
Supporting Features	Black box and non-functional tests
Testing & Refinement	Full 204-test suite execution, edge case validation

Implementation:

Overview

INTERCEPTOR is built as a distributed four-service system. Each service runs independently and communicates over HTTP. The frontend (Next.js, port 3000) and admin panel (Next.js, port 3001) only ever communicate with the Express backend (port 5000). The Express backend communicates with the Fast API AI service (port 8000). The AI service communicates with Olema (port 11434), which runs the language model locally. The frontend never has direct access to the AI layer - the Express backend acts as the single gateway.

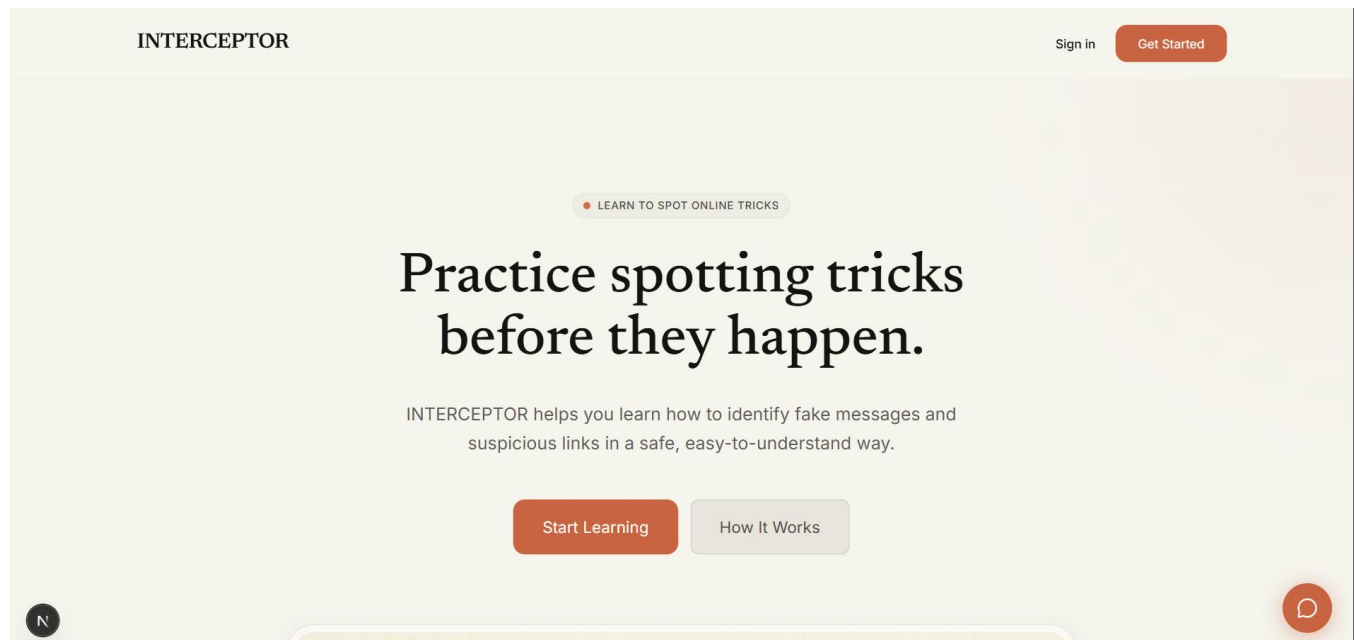


Figure 1 Landing page -01

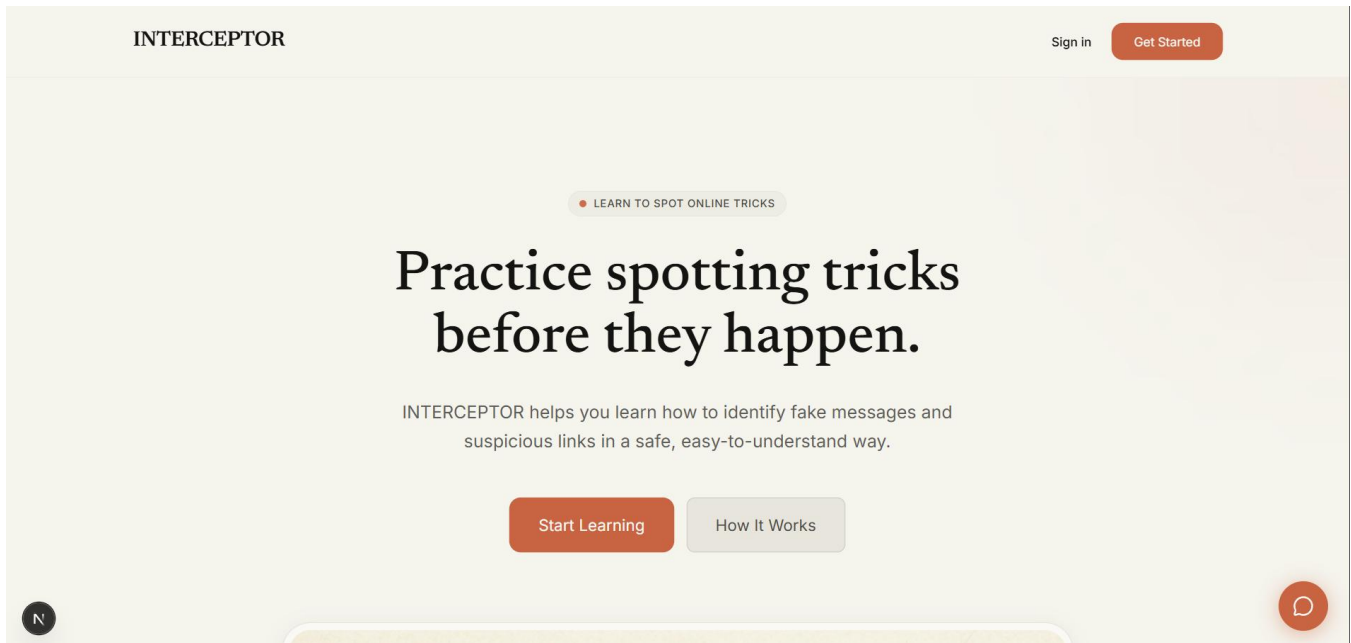


Figure 2 Landing Page 02

1. Frontend - User Interface

The user-facing frontend is built with Next.js 14 using the App Router and styled with Tailwind CSS. It is structured around two types of pages: public-facing pages (landing, login, register, contact) and authenticated pages accessible only after login.

Authenticated pages use a shared layout wrapper that includes sidebar navigation. Pages like the simulation screen, learning modules, phishing lab, and history sit inside this layout. The landing page has its own layout with a header, footer, and the integrated chatbot widget.

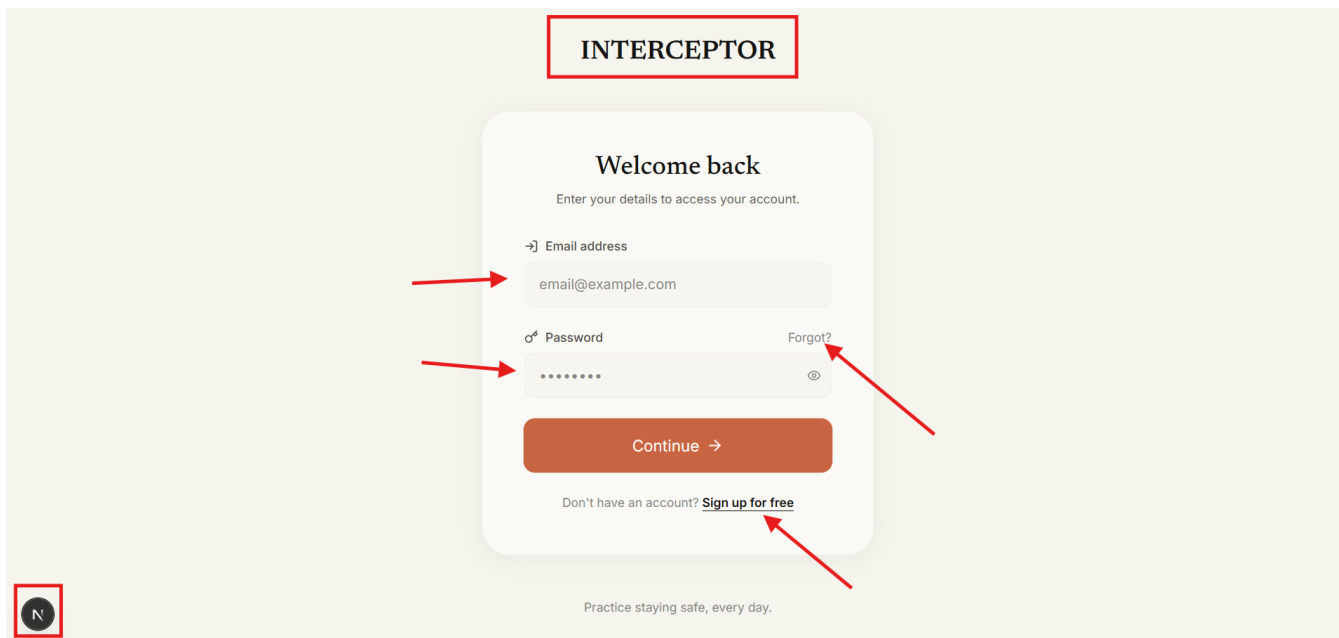


Figure 3 User Login

The simulation screen is the most complex frontend component. It maintains local state for the conversation history, the current attacker message, and the streaming response. When the user submits a reply, the frontend opens a fetch stream to the backends' SSE endpoint and appends each received token to the displayed message in real time, producing a typewriter effect that makes the AI attacker's responses feel live and immediate.

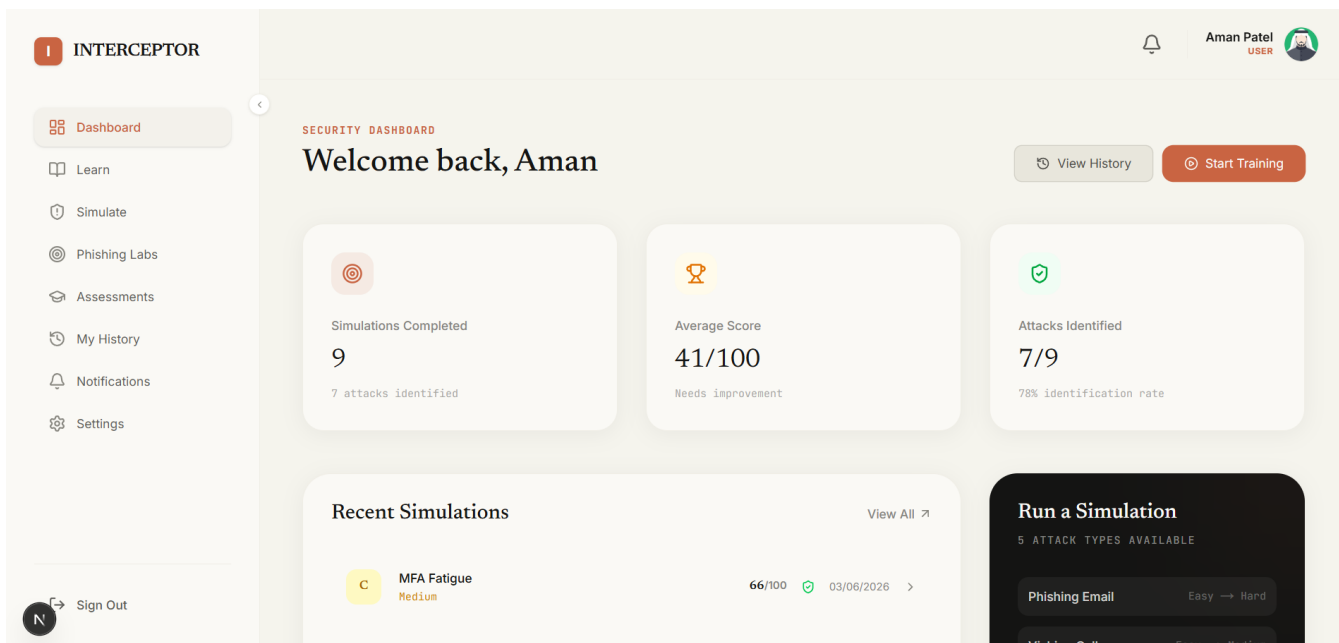


Figure 4 User Dashboard

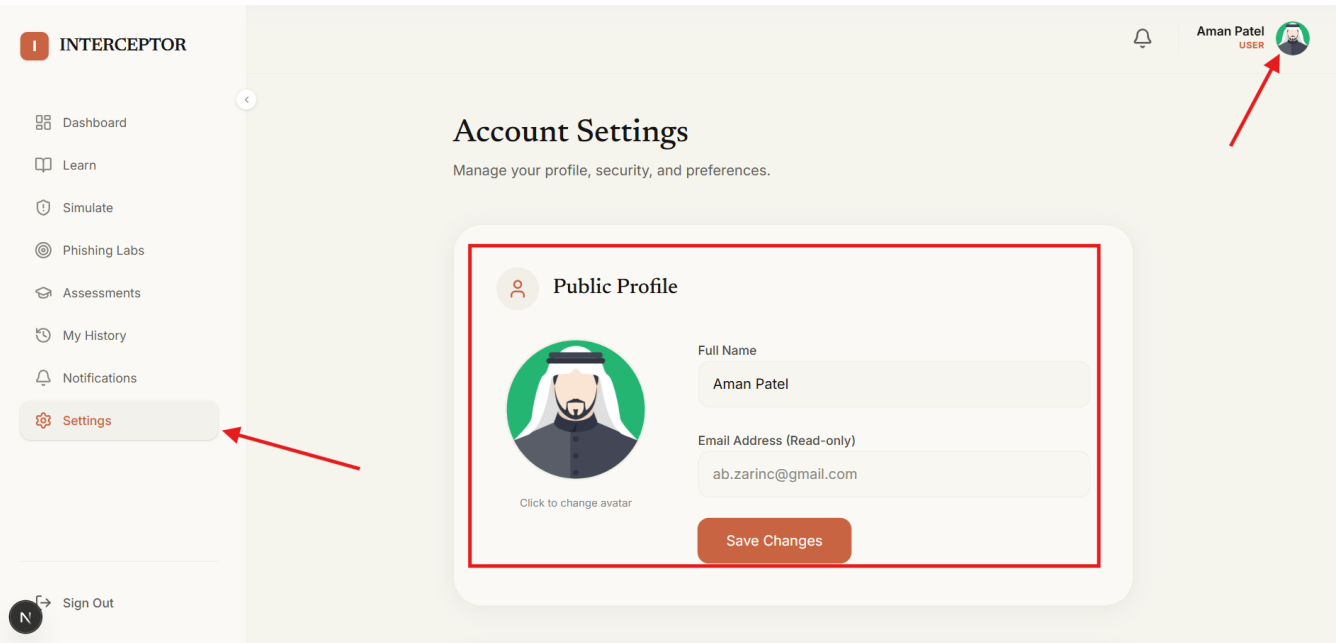


Figure 5 User Settings Page

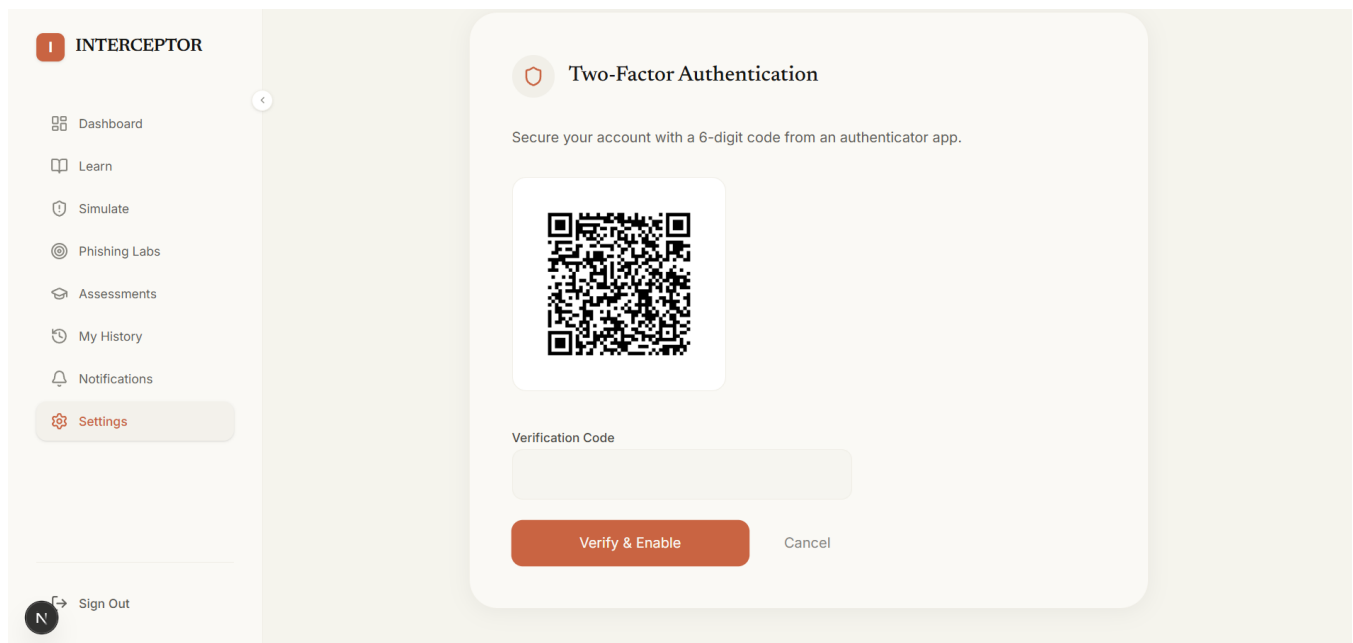


Figure 6 TOTP Implementation Using Google Authenticator

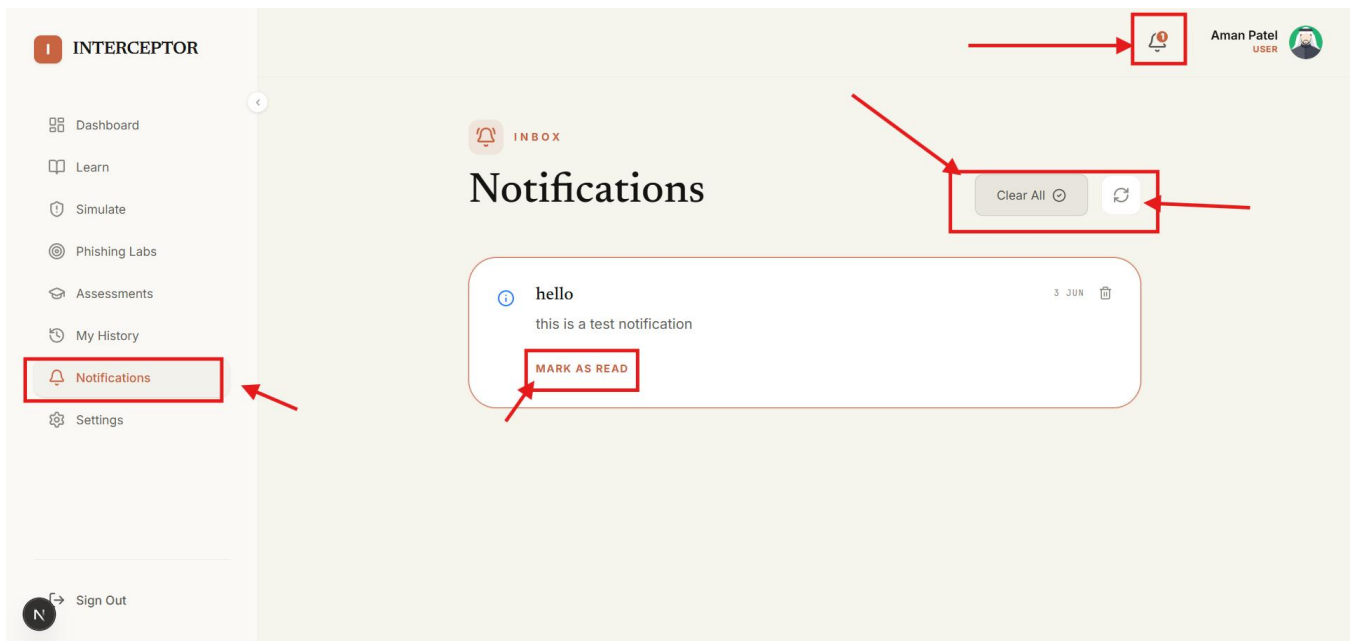


Figure 7 User Notifications Management

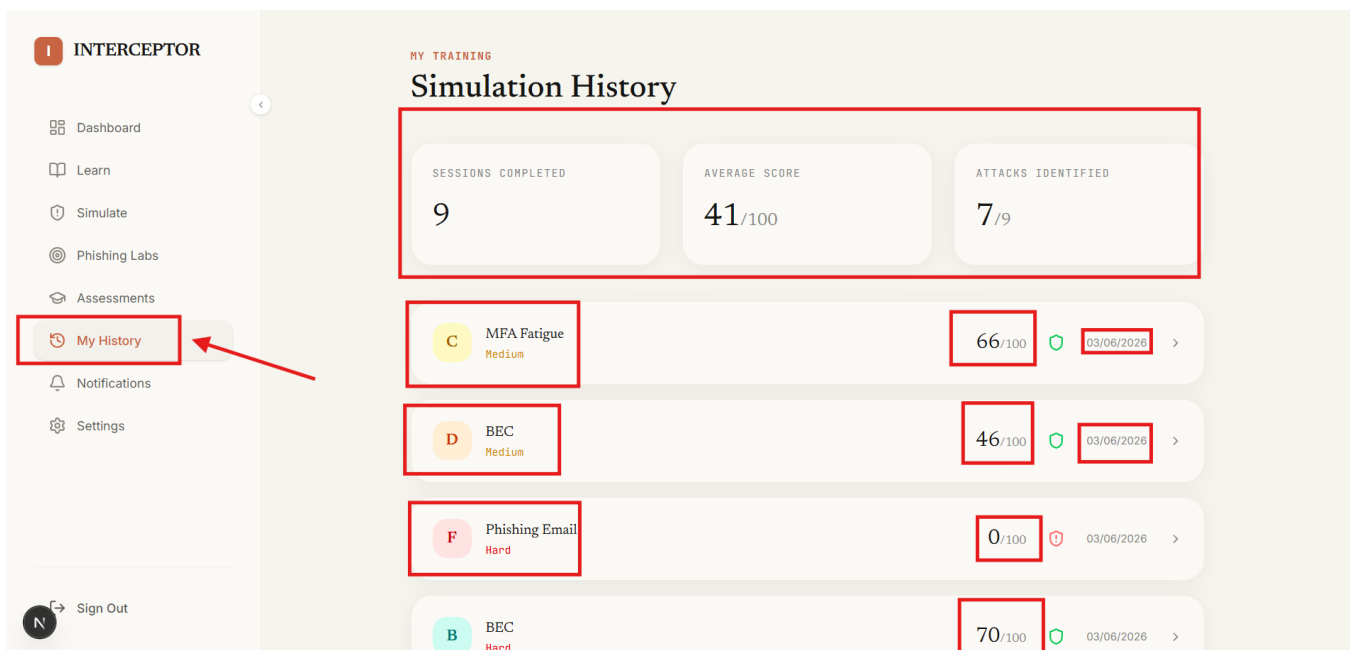


Figure 8 Simulation History

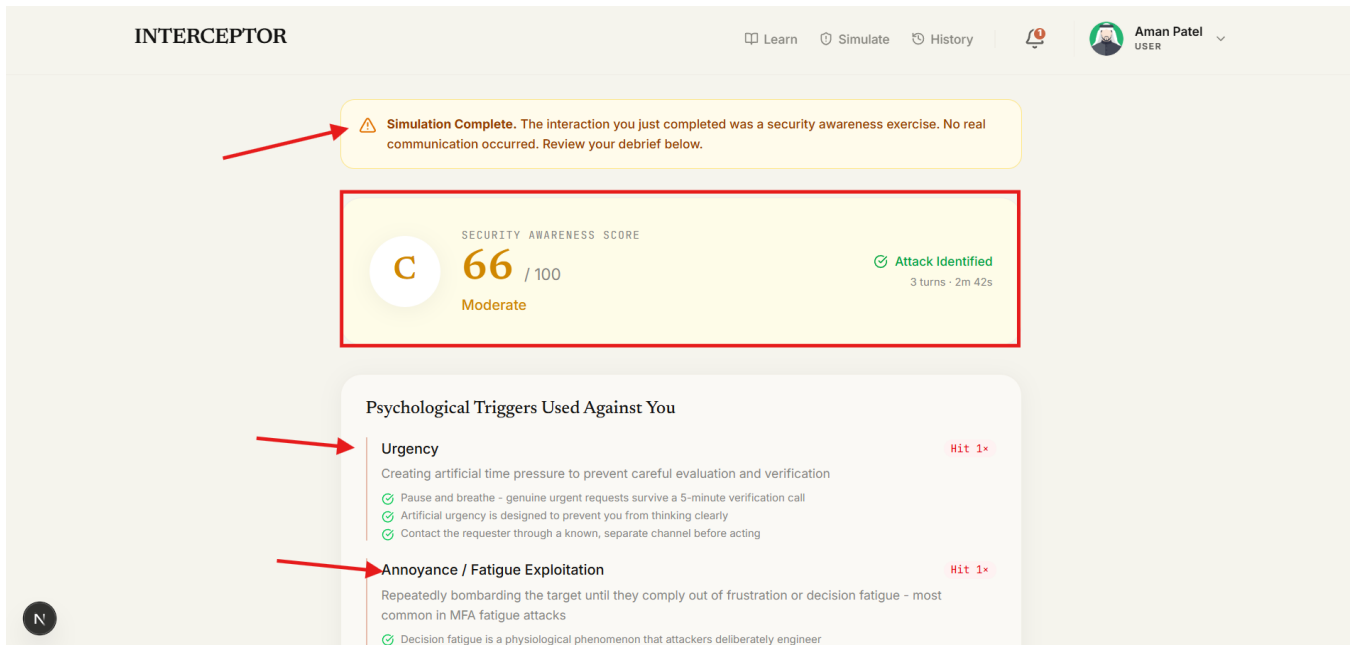


Figure 9 Detailed View of a Particular Simulation History

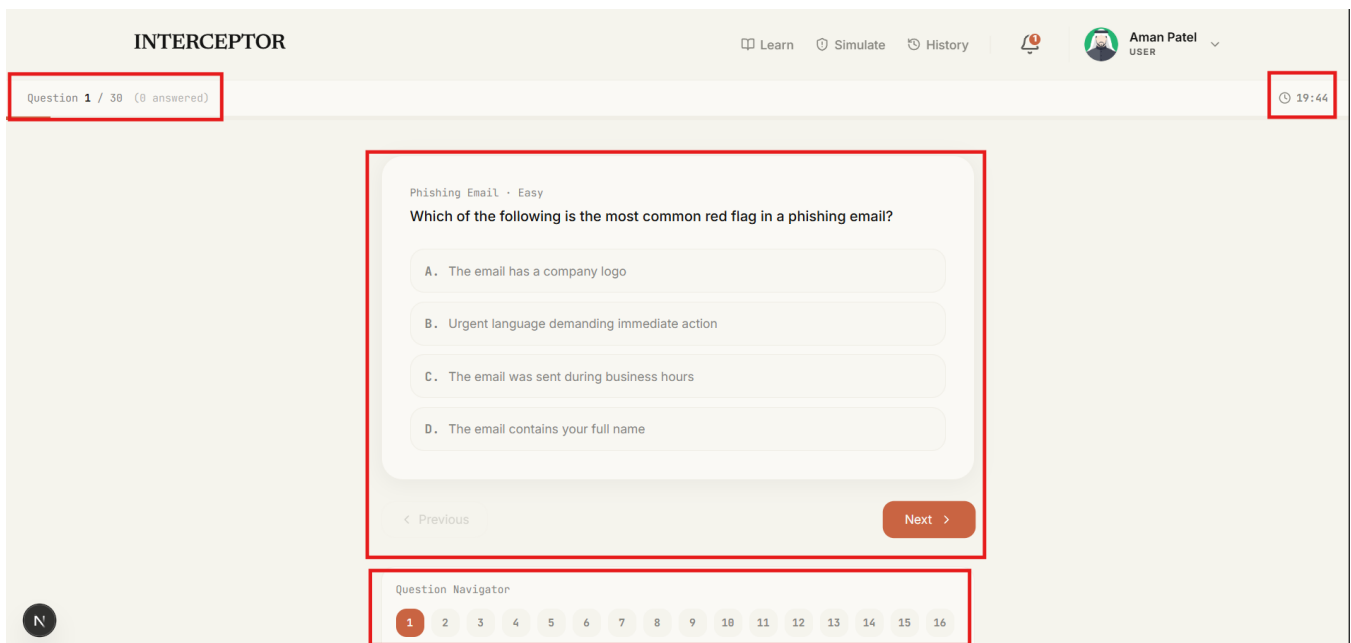


Figure 10 Time Based Security Assessment

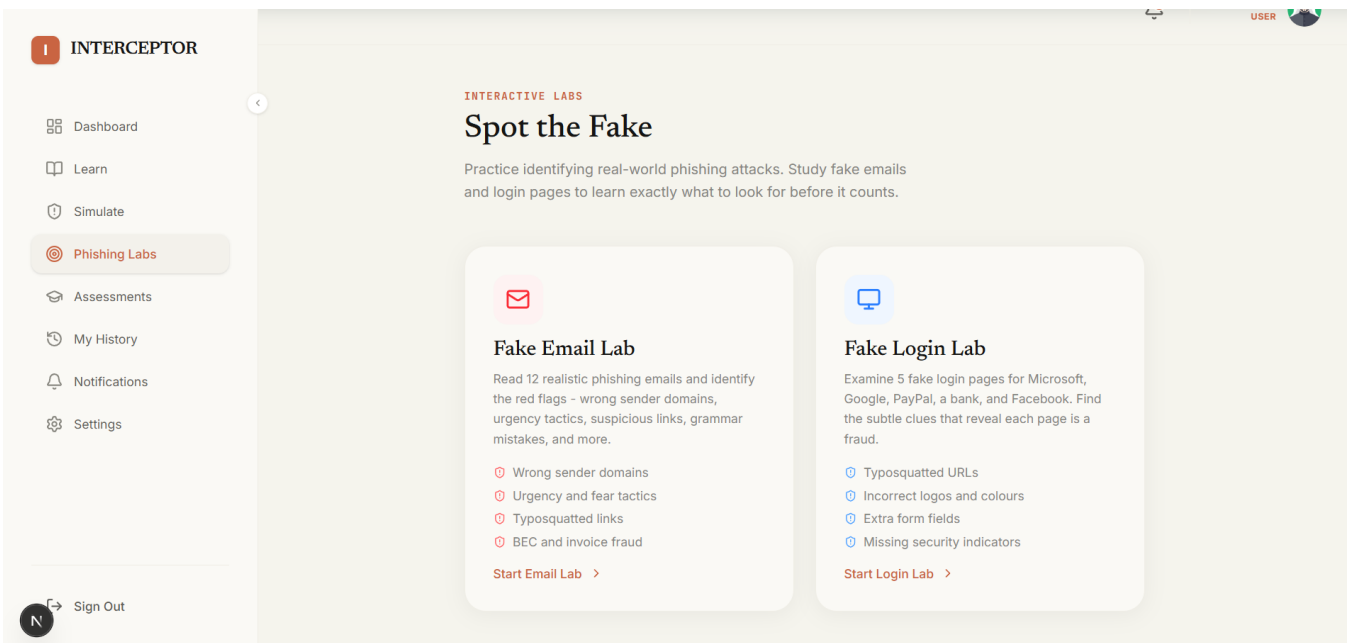


Figure 11 Phishing Labs



Figure 12 Sample Simulation

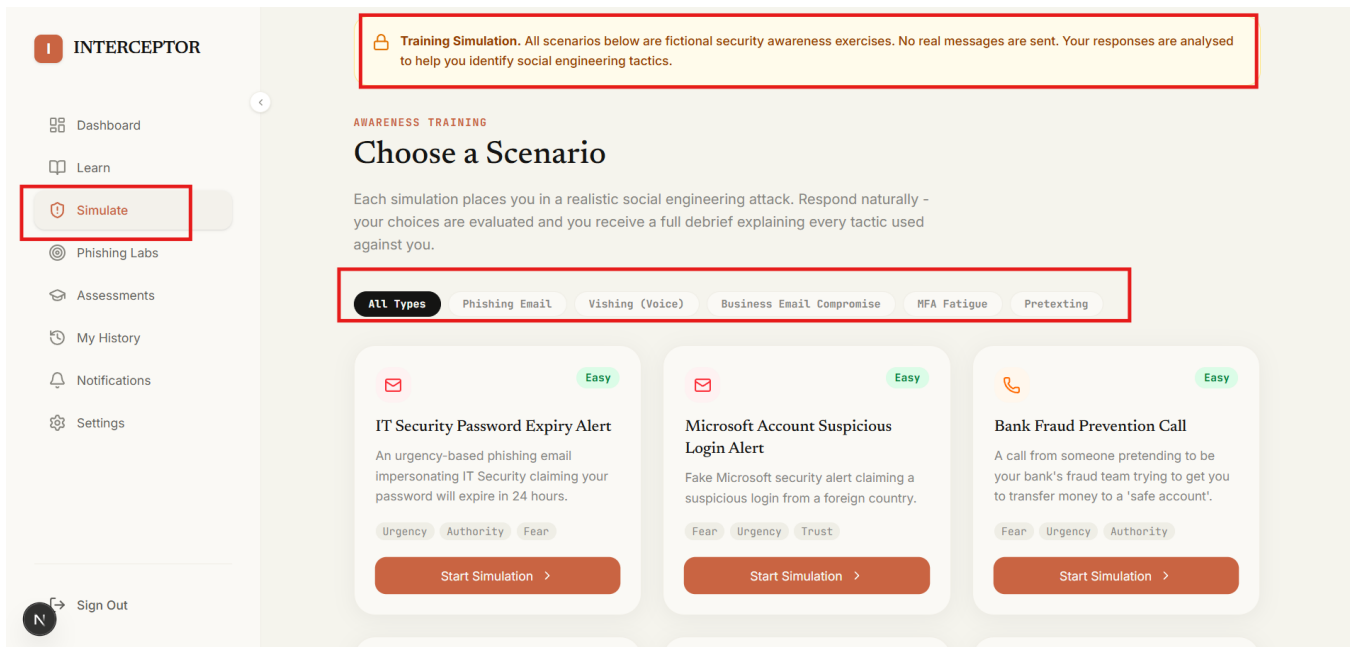


Figure 13 AI Powered Simulations

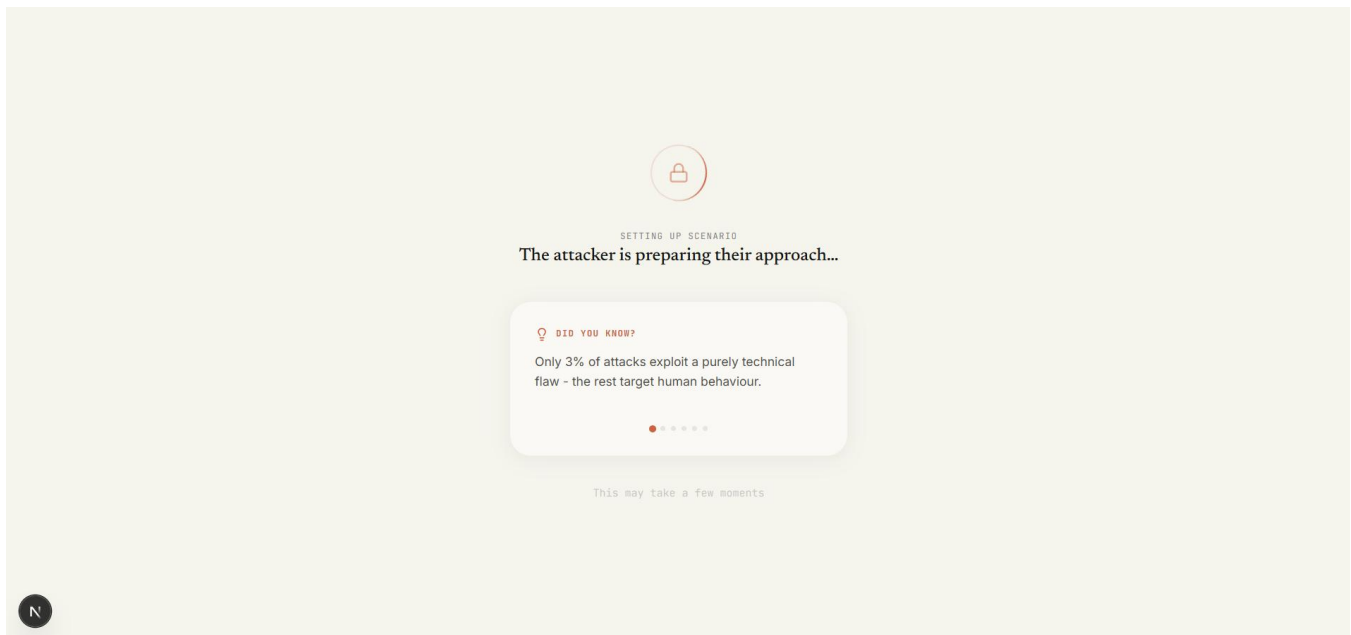


Figure 14 Attack Scenario getting prepared

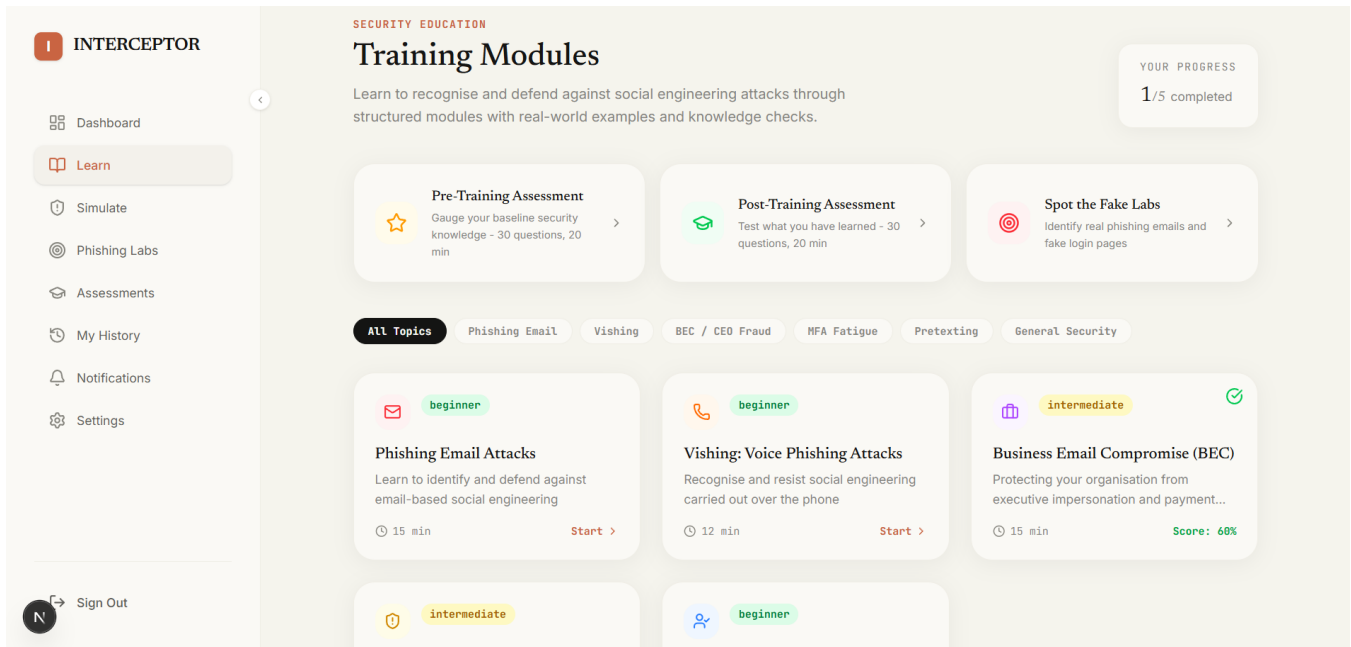


Figure 15 Training Modules (Learning Material)

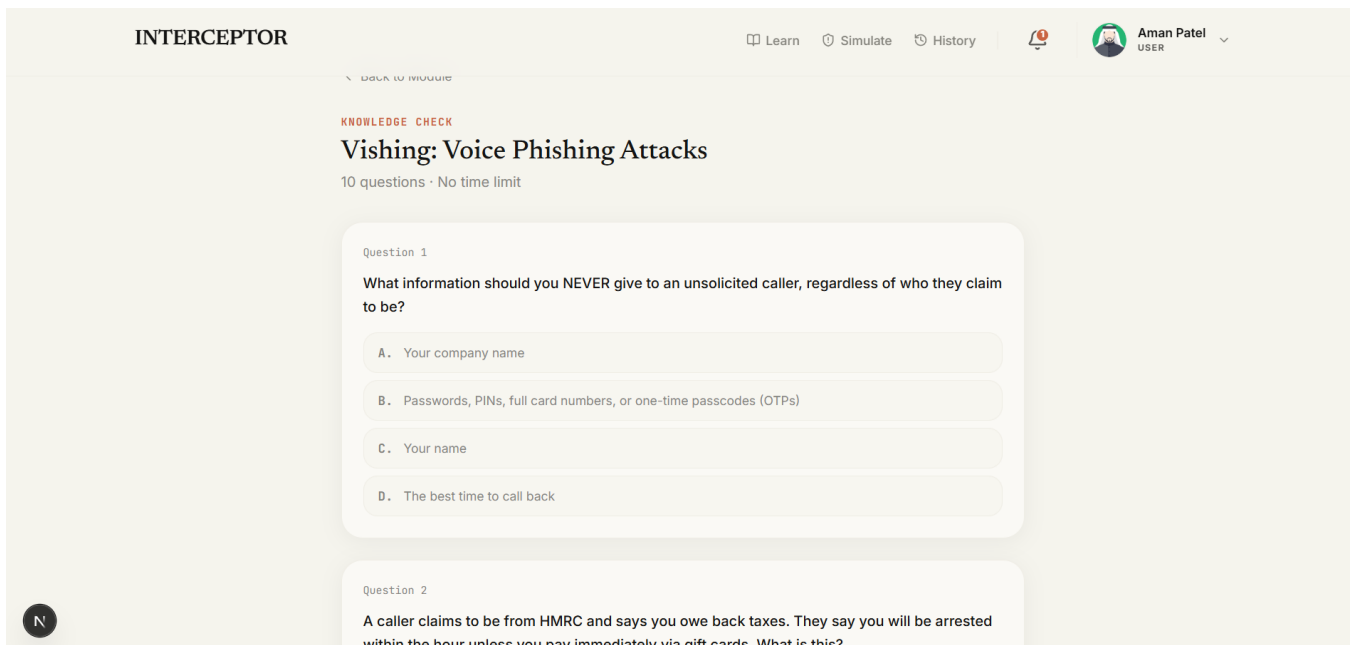


Figure 16 Knowledge check at the end of each module

The admin panel is a completely separate Next.js application on port 3001. It connects to the same Express backend API but uses admin-only routes. Admins can view all registered users, manage scenarios and personas, read contact form submissions, send notifications, and access platform-wide analytics presented as charts and KPI cards.

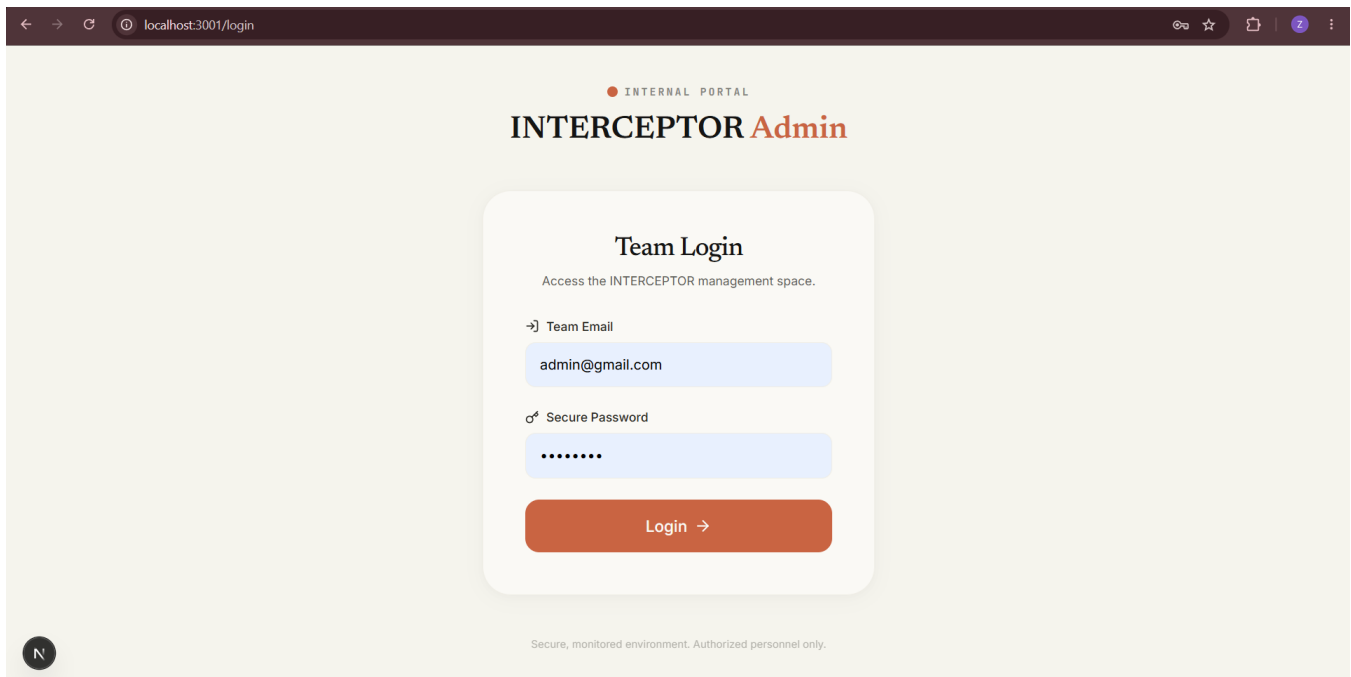


Figure 17 Admin Login Page

2. Express Backend - API Layer

The Express backend is the system's central gateway. It handles all authentication, enforces access control, manages database reads and writes, and coordinates with the AI service.

Middleware Stack:

Every request passes through a layered middleware stack in order: express. Json parses the body, cookie Parser makes cookies available, helmet sets security headers (preventing clickjacking, XSS reflection, and MIME sniffing), and cores restricts requests to the configured frontend origins. The authentication rate limiter sits on top of the /API/v1/auth routes and permits a maximum of 50 requests per IP per 15-minute window, blocking brute-force login attempts.

Route Architecture:

The API is versioned under /API/v1/ and divided into focused route files:

Route	Purpose
/auth	Registration, login, logout, email verification, 2FA, password reset
/simulation	Start, respond, stream, end simulation sessions
/training	Learning modules, quiz submission, progress tracking
/phishing	Phishing lab templates
/scenarios	List available simulation scenarios
/analytics	Platform-wide statistics (admin only)
/admin	User and scenario management (admin only)
/chat	Landing page chatbot (SSE stream)

Authentication Middleware:

The protect middleware validates every protected request. It reads the JWT from either the HTTP Only cookie (set at login) or the Authorization: Bearer header. It verifies the token's signature and expiry using jitterily, then fetches the current user from MongoDB and attaches them to request. The authorize() factory middleware then checks req.user.role against the required roles - any JWT that does not carry the admin role is rejected with a 403 before reaching admin route handlers.

3. Authentication System

Registration and Email Verification:

When a user registers, the system validates the submitted data, creates a User document in MongoDB with the password bcrypt-hashed at cost factor 12, and generates a 32-byte random verification token. The raw token is sent to the user's email as a clickable link; only the SHA-256 hash of the token is stored in the database. This means that even if an attacker gains read access

to the database, they cannot use the stored hash to verify an account - they would need the original token that was emailed.

The user cannot log in until email verification is confirmed. When the user clicks the link, the frontend sends the raw token to the backend, which hashes it, finds the matching user, and sets `isEmailVerified = true`.

Login and JWT Issuance:

On login, the system retrieves the user, compares the submitted password against the stored bcrypt hash, checks that the email is verified, and issues a JWT signed with the server's secret key. The token is set as an `HttpOnly` cookie, meaning JavaScript running in the browser cannot read it. This eliminates the most common attack against stored tokens - XSS. The `sameSite: Lax` cookie attribute prevents cross-site request forgery.

Two-Factor Authentication:

INTERCEPTOR implements TOTP (Time-based One-Time Password), the same standard used by Google Authenticator. On setup, the server generates a base32 secret and returns a QR code. When the user scans it with their authenticator app and confirms a valid 6-digit code, the secret is permanently stored. From that point forward, every login requires a valid TOTP code after the password - the JWT is only issued after both factors pass.

Password Reset:

Password reset uses the same hash-only token pattern as email verification, but with a 10-minute expiry. The raw token travels only in the email link; the database stores only the hash.

4. Simulation Engine

The simulation is the core feature of INTERCEPTOR. It is implemented across the Express backend and the FastAPI AI service, with three distinct phases.

Phase 1 - Starting a Session:

When the user selects a scenario and clicks Start, the Express backend looks up the scenario in MongoDB, retrieves the user's profile (job role, department, assessed vulnerability tags), and sends this data to the AI service's `/simulate/start` endpoint.

The AI service creates an in-memory session with a UUID, then runs the initial RAG retrieval to fetch relevant attack tactics. The retrieved tactics are injected into a prompt along with the attacker persona, scenario type, and the user's profile context. Gemma 7B generates the first

attacker message. The system scans the generated message for psychological triggers using regex pattern matching. The complete result - session ID, attacker message, triggers fired - is returned to Express, which creates a SimulationSession document in MongoDB and returns the attacker message to the frontend.

Phase 2 - Turn-by-Turn Conversation:

When the user sends a reply, Express verifies the session belongs to the logged-in user and forwards the response to the AI service.

The AI service runs a two-stage evaluation of the user's reply. The first stage is rule-based: regex patterns immediately classify the response as compliant, suspicious, or identifying the attack. This produces a fast initial score and a reaction hint. The second stage runs an LLM evaluation in a parallel thread - it sends the attacker message, user response, and trigger list to Ollama at near-zero temperature (0.1) and asks it to return a structured JSON assessment. The two results are merged, with the LLM score weighted at 60% because it understands nuance that regex cannot.

Meanwhile, the main thread runs the next retrieval cycle. If the user showed suspicion, escalation tactics are retrieved. If the user seemed compliant, persuasion tactics are retrieved to continue the pressure. A new attacker message is generated and streamed token-by-token back to Express via SSE. Express pipes each token directly to the browser, producing the typewriter effect. When streaming ends, Express updates the session in MongoDB with the user's response, the evaluation scores, and the new attacker message.

Phase 3 - Session End and Scoring:

When the session ends (manually by the user, or when the turn limit is reached), Express calls the AI service's /simulate/end endpoint. The service computes the overall session score - an average of per-turn scores on a 0–100 scale - and assigns a letter grade (A through F). It then runs a final RAG retrieval specifically for debrief content, fetching explanations for each psychological trigger that fired during the session. These explanations include what the trigger is, how it was used, and specific counter-measures the user can apply in real situations.

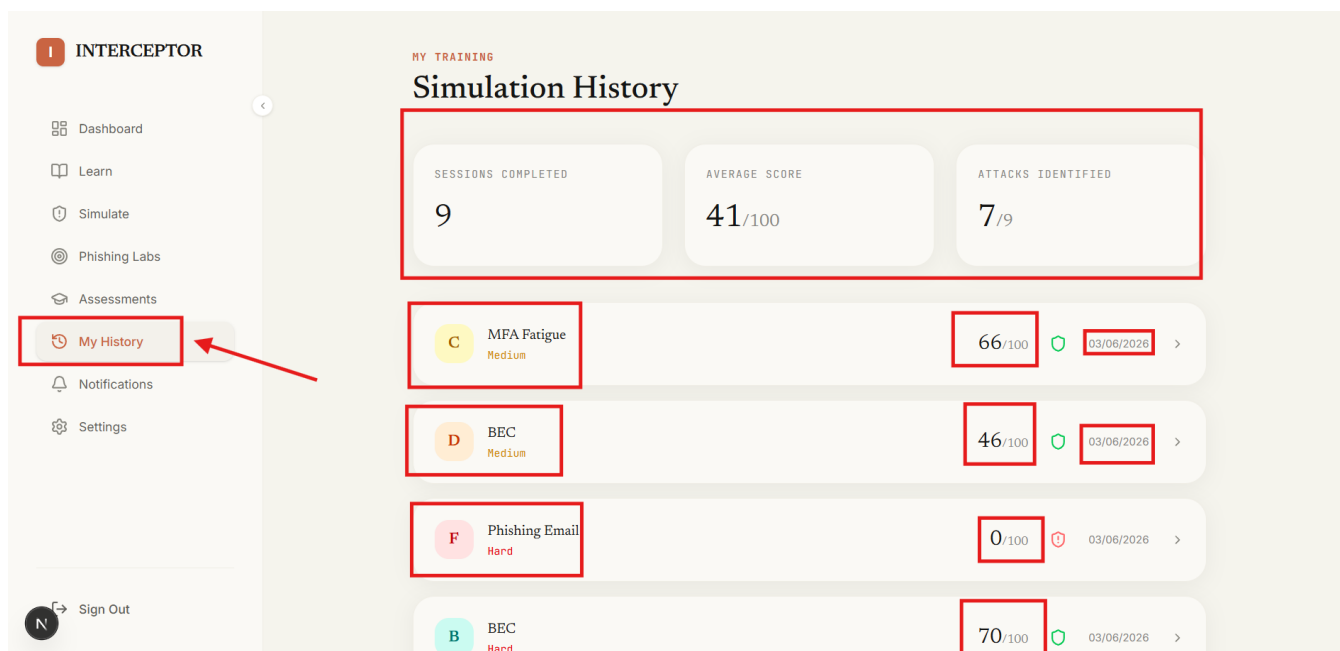


Figure 18 Score Based Feedback

All of this is saved to the SimulationSession document in MongoDB: the score, grade, trigger breakdown, debrief content, and a status of "completed". The frontend displays the results screen with the score, grade, a turn-by-turn breakdown, and personalised advice.

5. RAG Pipeline

The RAG (Retrieval-Augmented Generation) pipeline is what makes the attacker's messages grounded in real social engineering knowledge rather than generic AI output.

Knowledge Base:

The knowledge base is stored as JSON files covering each attack scenario (phishing, vishing, BEC, MFA fatigue, pretexting) and a file of psychological triggers (urgency, authority, social proof, scarcity, liking, reciprocity). During setup, a seeding script reads these files, splits them into chunks, and uploads them to Pinecone with metadata tags for scenario_type and chunk_type.

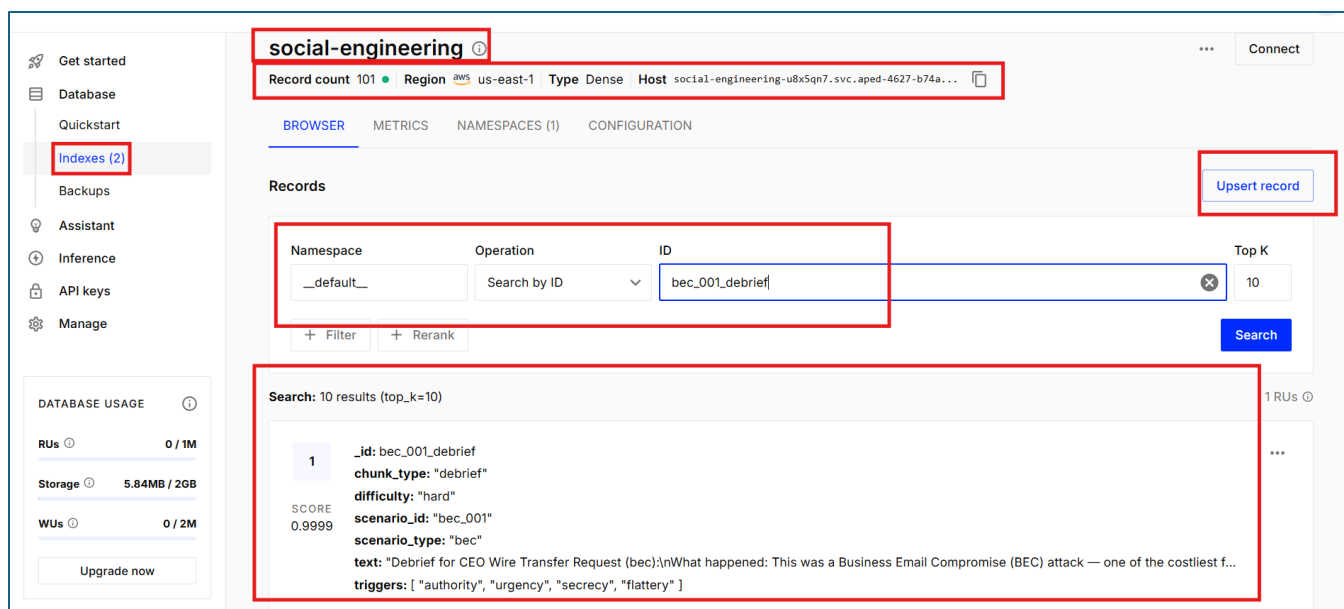


Figure 19 RAG DB (PINECONE)

Embedding and Retrieval:

At retrieval time, the system constructs a natural language query describing what it needs (for example, "how to start a phishing attack targeting a finance employee"). The BGE embedding model (BAAI/bge-large-en-v1.5) converts this query into a 1024-dimensional vector. Pinecone performs approximate nearest neighbour search, returning the 10 most semantically similar stored chunks filtered by scenario type and attack stage.

Reranking:

The top 10 candidates from Pinecone are then passed to a cross-encoder reranker (ms-marco-MiniLM-L-6-v2). Unlike the embedding model which encodes query and document separately, the cross-encoder reads both together and produces a true relevance score. The top 3 results after reranking are injected into the LLM prompt. This two-stage approach - fast approximate retrieval followed by accurate reranking - is standard industry practice for production RAG systems.

6. Learning Modules and Phishing Lab

The Learn section provides five structured training modules covering phishing, smishing, ransomware, password safety, and safe browsing. Each module is stored in MongoDB with reading

sections and a bank of quiz questions. The ModuleProgress model tracks per-user completion and quiz scores, using MongoDB's upsert operation so progress is updated rather than duplicated on repeat attempts.

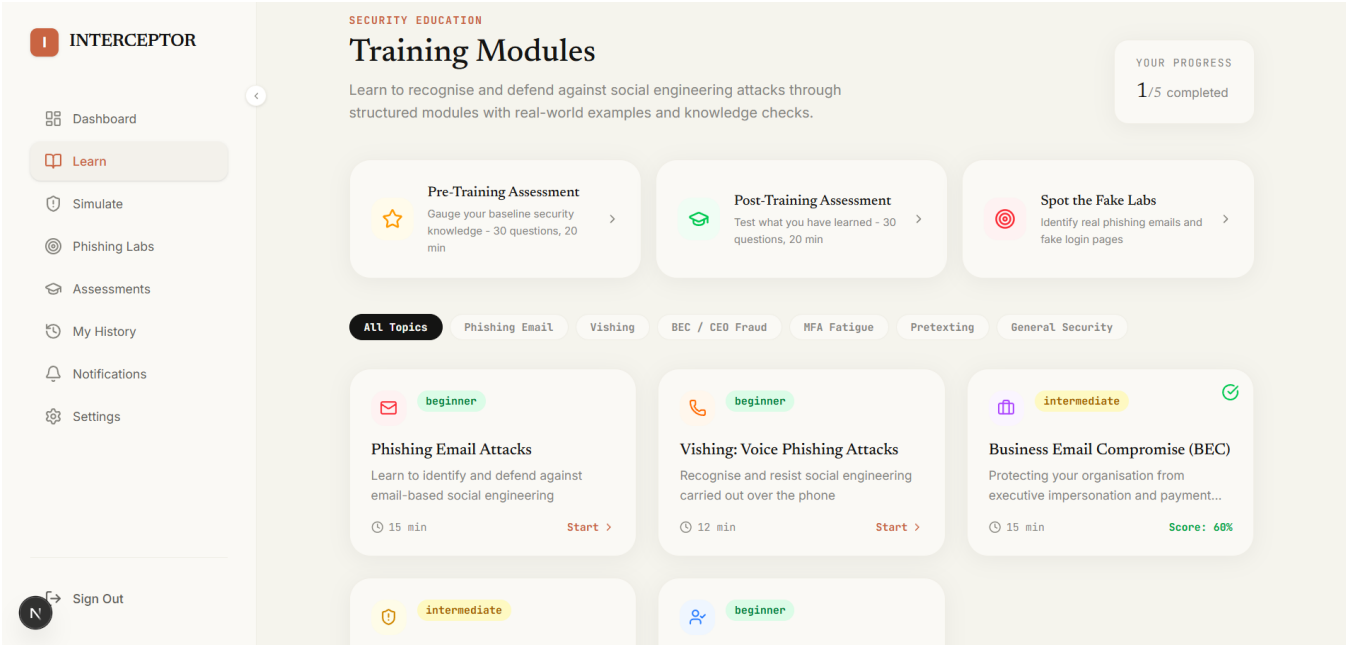


Figure 20 Learning Modules

Quiz questions are served by random sampling from the question bank using MongoDB's \$sample aggregation operator, ensuring users get a different selection each time rather than a predictable fixed set.

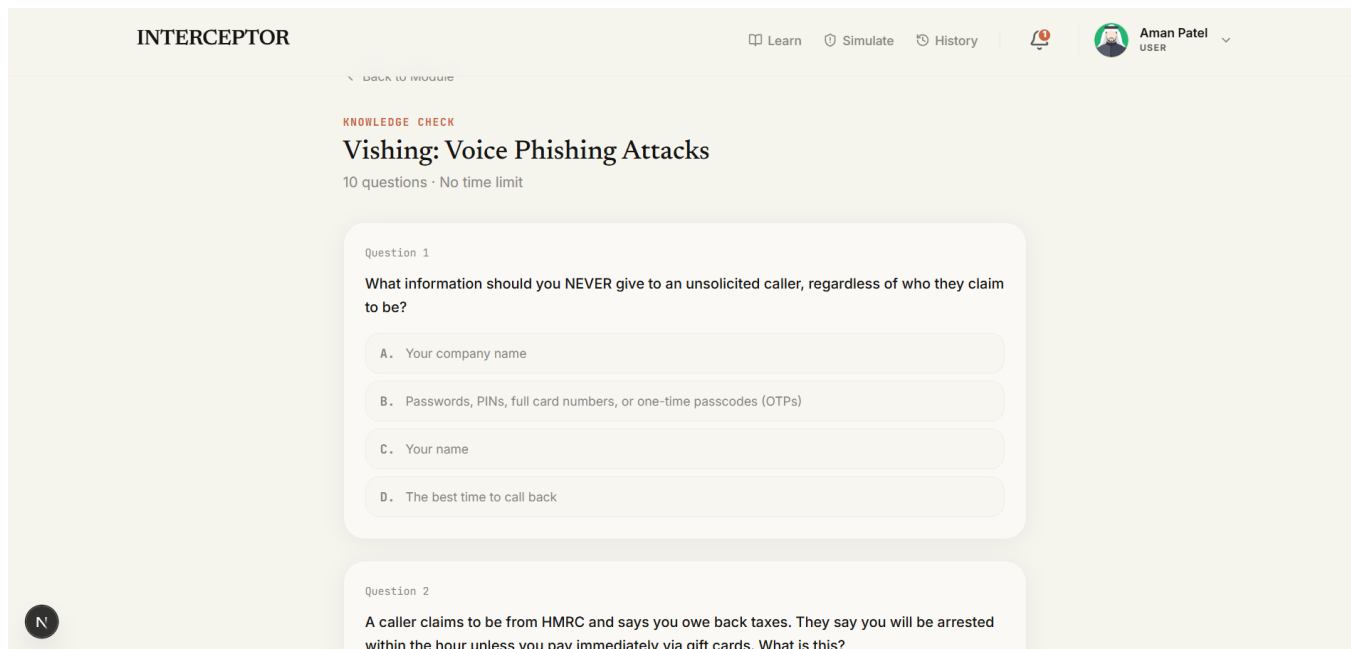


Figure 21 Quiz at End

The Phishing Lab presents users with realistic fake phishing emails and cloned login pages with intentional red flags embedded. These are static templates seeded from JSON files, rendered in the frontend so users can practise identifying the signs of a phishing attempt before encountering one in a simulation.

7. Scoring and Debrief

Every simulation session produces a structured score report. The overall score (0–100) represents the user's security behaviour across the session - higher scores mean the user resisted more effectively. The grade (A–F) maps directly to score bands: A is 85 and above, B is 70–84, C is 55–69, D is 40–54, and F is below 40.

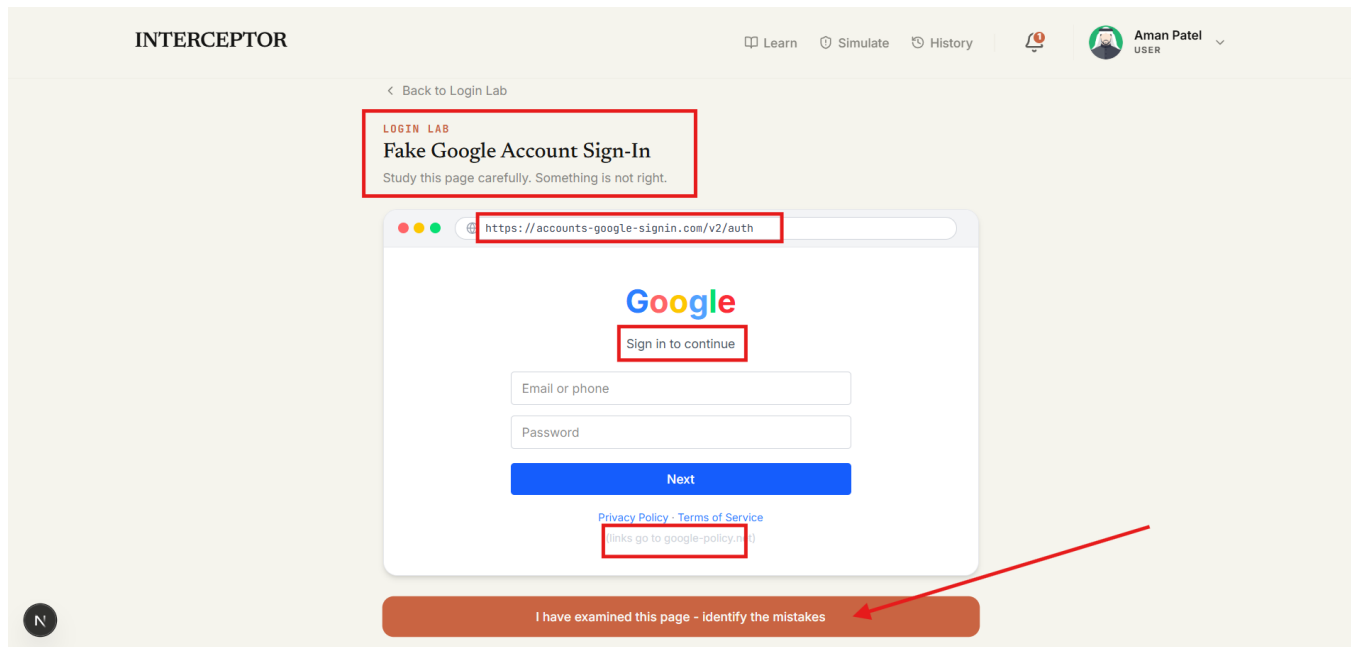


Figure 22 Fake Simulation

The debrief is generated by retrieving knowledge chunks specifically tagged for debriefing the triggers that fired during the session. Each trigger explanation tells the user what the psychological technique was (for example, urgency or authority), how many times it appeared in the session, and what to do differently when they encounter it in real life. This personalised feedback is the key differentiator from traditional awareness training - the advice is specific to what actually happened in that user's session.

8. Admin Dashboard

The admin panel provides a separate management interface. Administrators can view all registered users, their session history, and platform-wide statistics including total registrations, simulation completion rates, and score distributions. Scenarios and personas can be created, edited, or deactivated without touching the codebase - they are stored as MongoDB documents that the frontend reads at runtime.

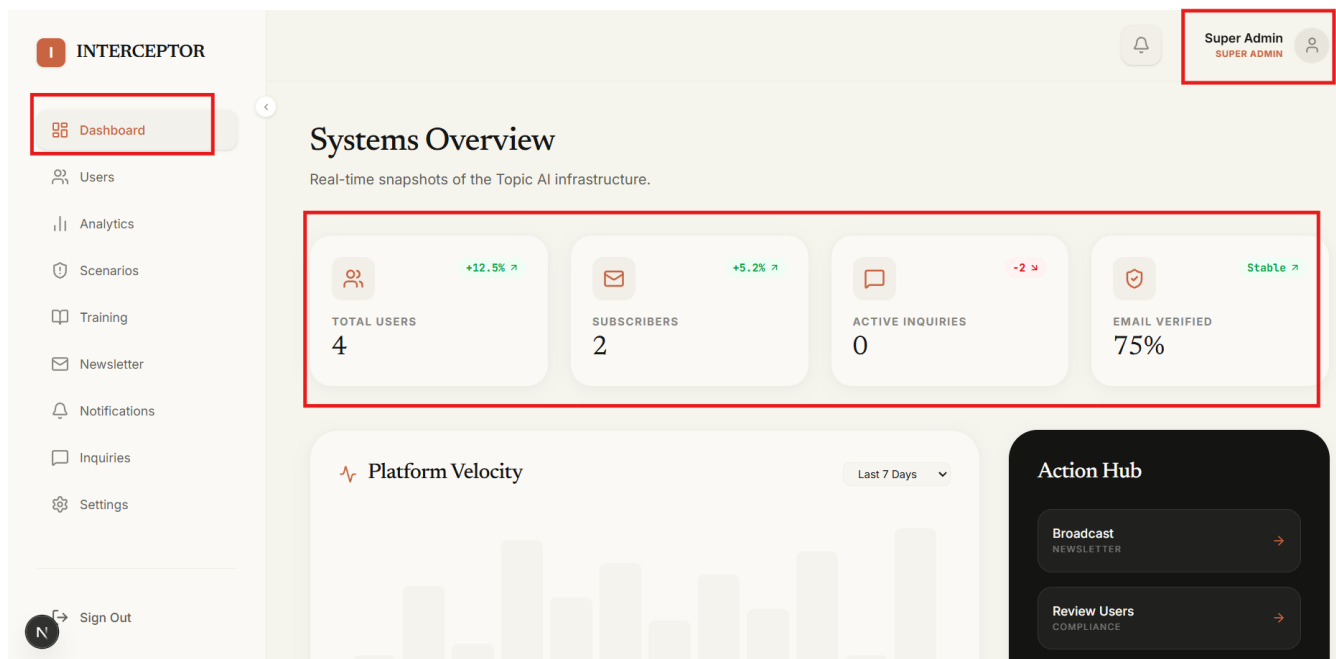


Figure 23 Admin Dashboard

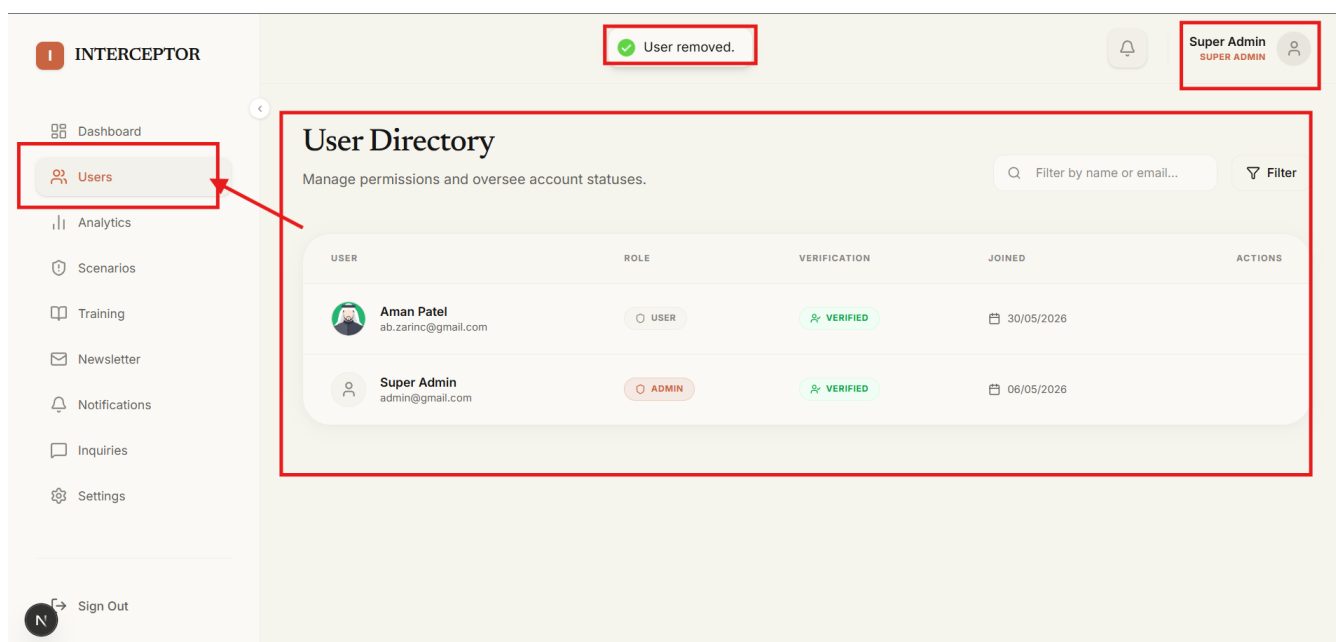


Figure 24 User Management

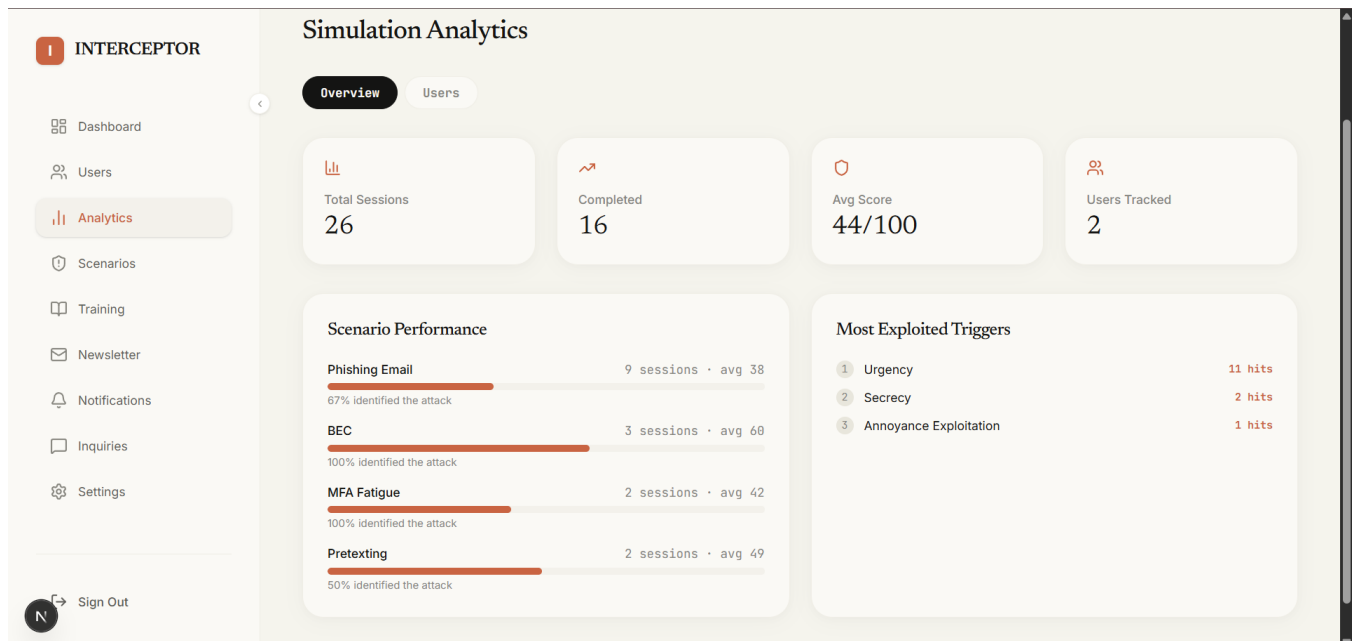


Figure 25 Platform Analytics

INTERCEPTOR

Scenario Management

Seed Defaults + New Scenario

NAME	TYPE	DIFFICULTY	PERSONA	STATUS	
Third-Party Auditor Access	pretexting	hard	External security auditor	Inactive	
New Employee Onboarding Pretext	pretexting	medium	New employee or intern	Active	
MFA Push Notification Bombing	mfa fatigue	medium	IT Support (after credential theft)	Active	
Vendor Payment Details Change	bec	medium	Trusted external supplier	Active	
CEO Wire Transfer Request	bec	hard	Chief Executive Officer	Active	
HMRC Tax Compliance Call	vishing	easy	HMRC Compliance Officer	Active	
Bank Fraud Prevention Call	vishing	easy	Bank Fraud Prevention Officer	Active	
IT Help Desk Account Compromise Call	vishing	medium	IT Help Desk Technician	Inactive	
Shared Document Collaboration Request	phishing email	medium	Unknown colleague	Active	
Microsoft Account Suspicious Login Alert	phishing email	easy	Microsoft Security Team	Active	

Dashboard navigation: Dashboard, Users, Analytics, Scenarios, Training, Newsletter, Notifications, Inquiries, Settings, Sign Out.

Figure 26 Scenario Management

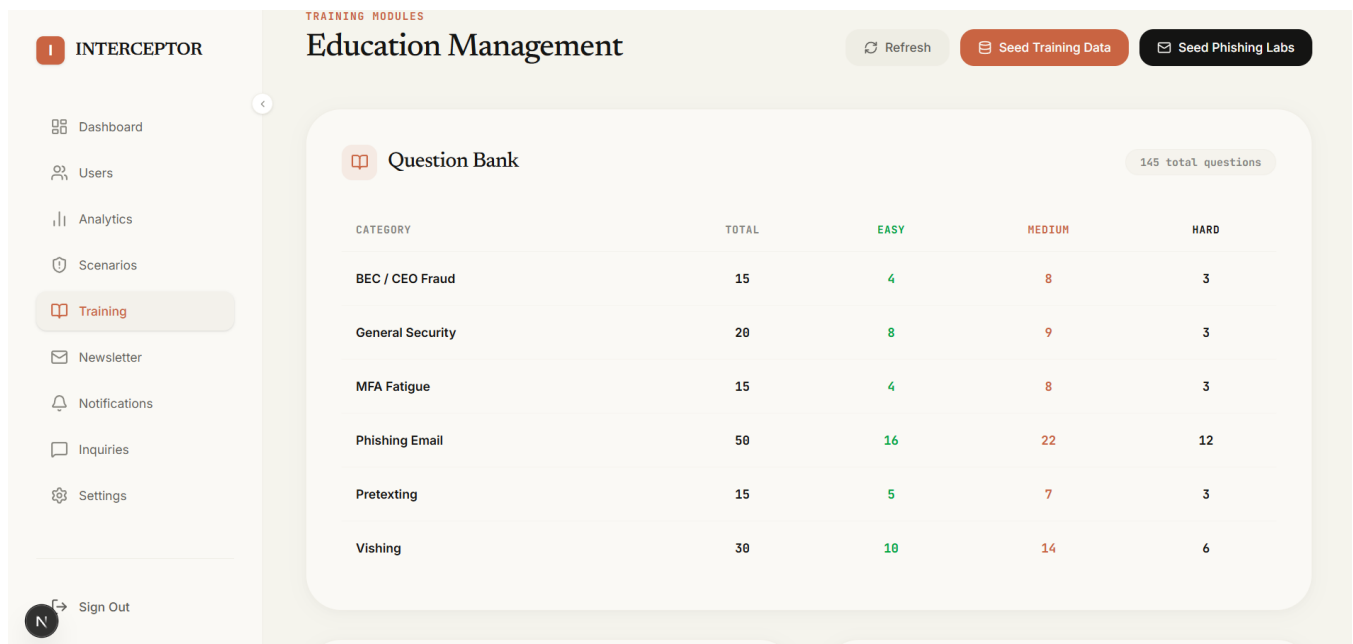


Figure 27 Training Modules

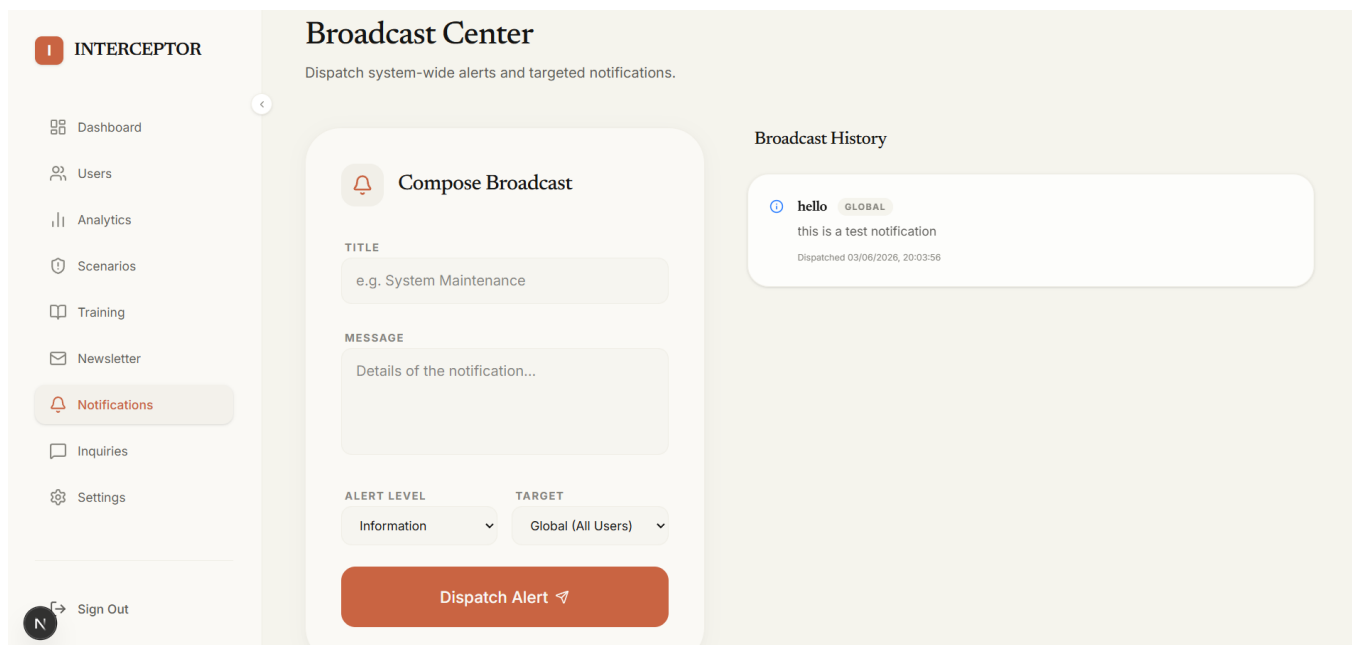


Figure 28 Broadcast Announcements

Analytics endpoints aggregate session data using MongoDB aggregation pipelines, grouping by scenario type, difficulty, time period, and score range.

Contact form submissions and newsletter signups are stored in dedicated collections and surfaced in the admin panel.

Admins can also send in-app notifications to individual users or all users, stored in the Notification collection and displayed in the user's frontend.

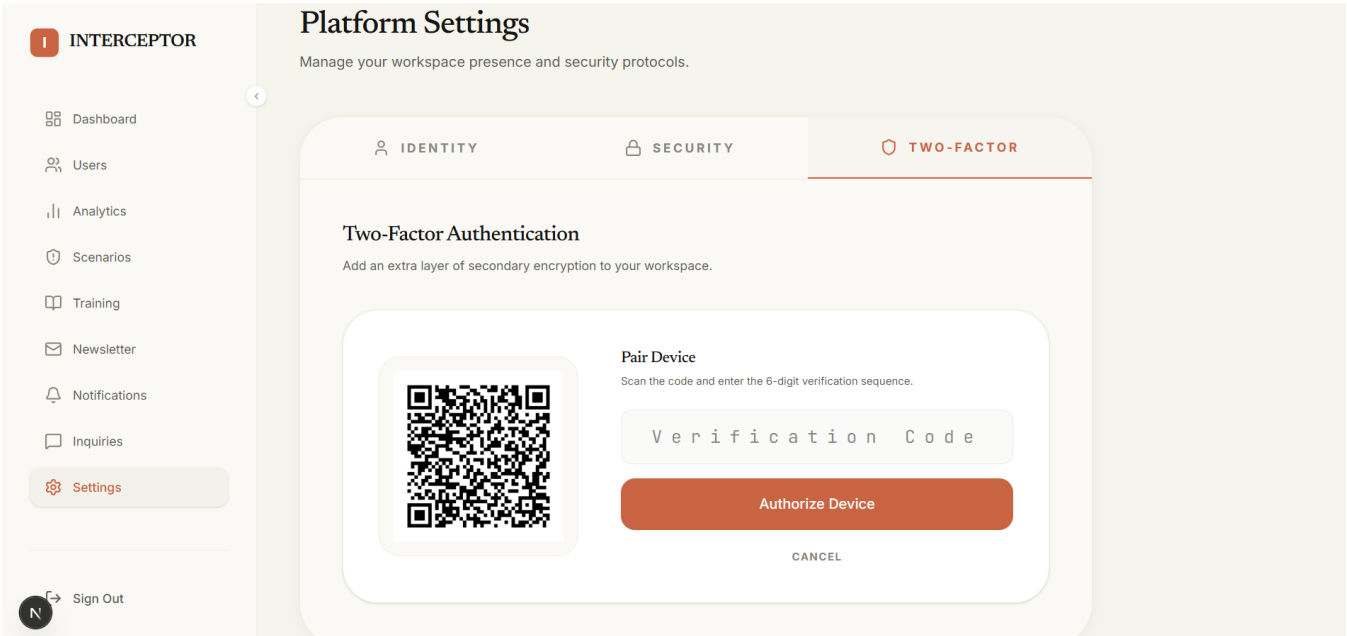


Figure 29 Profile Settings

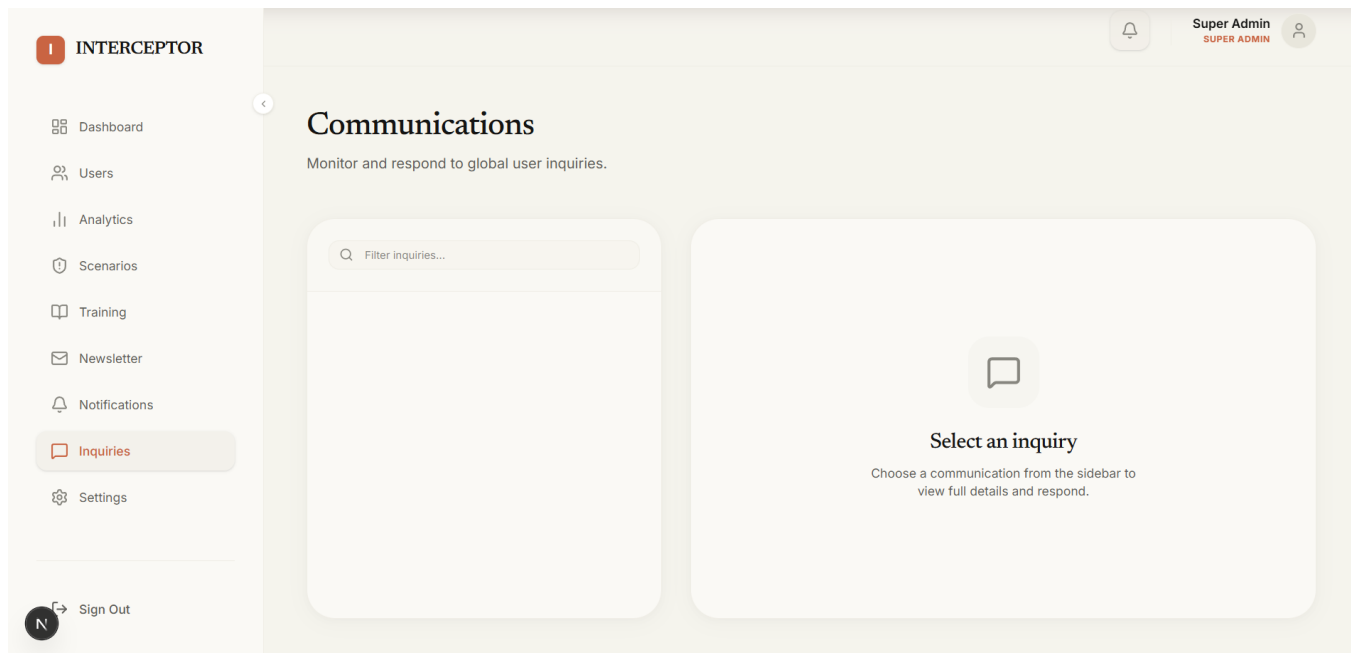


Figure 30 Contact Queries

9. Landing Page Chatbot

A chatbot widget is embedded on the public landing page to answer visitor questions about INTERCEPTOR. It operates as a streaming conversation backed by the AI service. User messages are sent to the Express backend, which checks them against a blocklist of off-topic patterns (code generation requests, model identity probing, prompt injection attempts) before forwarding to the AI service. Valid messages are answered by the LLM with a focused system prompt that constrains responses to INTERCEPTOR-related content only, streamed back token-by-token with the same SSE mechanism used in simulations.

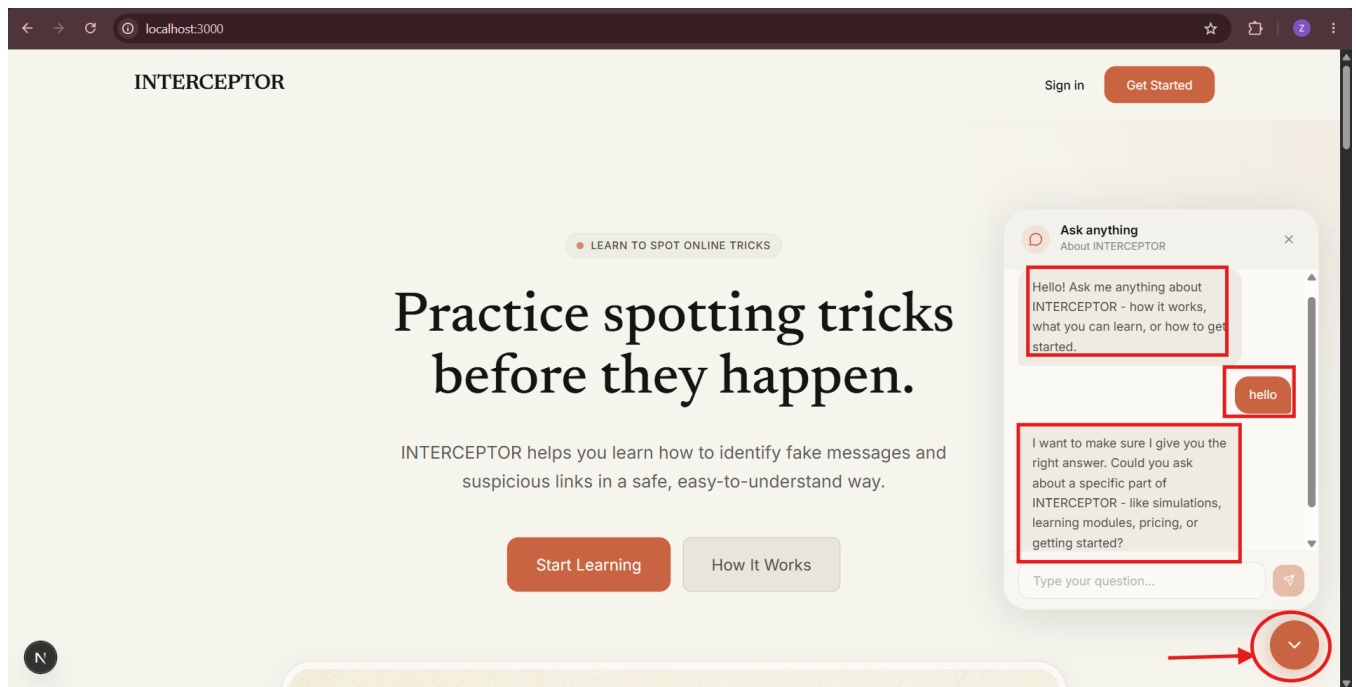


Figure 31 Landing Page Chatbot

Testing:

Overview:

Testing was conducted across five categories covering every layer of the system. A total of 204 tests were executed across JavaScript (Node.js / Jest) and Python (pytest) test suites. The table below summarises the results across all categories.

Category	Tests	Passed	Failed	Pass Rate
Unit Tests (JS)	45	45	0	100%
Integration Tests (JS)	18	18	0	100%
System / E2E Tests (JS)	16	16	0	100%
Black Box Tests (JS)	22	22	0	100%
Non-Functional Tests (JS)	19	19	0	100%
Unit Tests (Python)	58	58	0	100%
Integration Tests (Python)	26	19	0	73%*
TOTAL	204	197	0	97%

Test Environment: Windows 11, Node.js 22, Python 3.14, MongoDB in-memory (JS suites).

1. Unit Tests

1.1 Authentication — TC-01 to TC-05

File: BACKEND/TEST_unit_auth.test.js

Test ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC-01a	Malformed email — no @ symbol	notanemail.com	400, success:false	400	PASS
TC-01b	Email missing domain extension	user@domain	400, success:false	400	PASS
TC-01c	Empty email field	""	400, success:false	400	PASS
TC-01d	Duplicate email rejected	Existing email	400, success:false	400	PASS
TC-02a	Password shorter than 8 characters	Pass1!	400, success:false	400	PASS
TC-02b	Password with no uppercase letter	password1!	400, success:false	400	PASS
TC-02c	Password with no special character	Password1	400, success:false	400	PASS
TC-02d	bcrypt hash not equal to plaintext	Plain vs hash	hash ≠ plain, compare passes	✓	PASS
TC-02e	Password stored as bcrypt hash in DB	Valid register	Starts with \$2, compare passes	✓	PASS
TC-03a	Login with valid credentials	Correct email + password	200, JWT in cookie	200	PASS
TC-03b	JWT payload contains user ID and role	Valid login	decoded.id exists	✓	PASS

TC-03c	Login response does not expose password	Valid login	Password excluded from body	✓	PASS
TC-04a	Login with wrong password	Incorrect password	401, success:false	401	PASS
TC-04b	Login with unregistered email	Unknown email	401, success:false	401	PASS
TC-04c	Login with missing password field	No password	400, success:false	400	PASS
TC-04d	Login with empty credentials	{}	400, success:false	400	PASS
TC-05a	Forgot-password route reached	Registered email	200 (route hit)	✓	PASS
TC-05b	Reset token stored in DB	Forgot-password call	Token field set on User doc	✓	PASS
TC-05c	Reset token has future expiry	Forgot-password call	expire > now	✓	PASS
TC-05d	Invalid reset token rejected	Bad token	4xx response	✓	PASS
TC-05e	JWT issued with correct TTL	Valid login	TTL = 28800 s	✓	PASS

1.2 KPI Computation and Message Validation - TC-06, TC-12

File: BACKEND/TEST_unit_kpi.test.js

Test ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC-12a	Score 100 → Grade A	score = 100	"A"	"A"	PASS
TC-12b	Score 85 → Grade A (boundary)	score = 85	"A"	"A"	PASS

TC-12c	Score 84 → Grade B	score = 84	"B"	"B"	PASS
TC-12d	Score 70 → Grade B (boundary)	score = 70	"B"	"B"	PASS
TC-12e	Score 69 → Grade C	score = 69	"C"	"C"	PASS
TC-12f	Score 55 → Grade C (boundary)	score = 55	"C"	"C"	PASS
TC-12g	Score 54 → Grade D	score = 54	"D"	"D"	PASS
TC-12h	Score 40 → Grade D (boundary)	score = 40	"D"	"D"	PASS
TC-12i	Score 39 → Grade F	score = 39	"F"	"F"	PASS
TC-12j	Score 0 → Grade F	score = 0	"F"	"F"	PASS
TC-12k	Empty session → score 0, grade F	No turns	overall_score=0, grade="F"	✓	PASS
TC-12l	Single compliant turn (score 5) → F	score = 5	grade = "F"	✓	PASS
TC-12m	All refused turns (score 90) → A	score = 90	grade = "A"	✓	PASS
TC-12n	Mixed turns — correct average	100, 38, 5	avg = 48, grade D	✓	PASS
TC-12o	Null-score turns excluded from calc	Null turns	Only scored turns used	✓	PASS
TC-12p	attack_identified = true if any turn flags it	One flagged turn	true	✓	PASS
TC-12q	attack_identified = false if no turns flag it	No flagged turns	false	✓	PASS

TC-06a	CEO persona message passes validation	Valid CEO message	0 errors	✓	PASS
TC-06b	IT Support persona message passes validation	Valid IT message	0 errors	✓	PASS
TC-06c	Message with URL fails validation	Contains https://	URL detected	✓	PASS
TC-06d	Empty message fails validation	""	Empty detected	✓	PASS
TC-06e	Message over 500 characters fails	501-char string	Length exceeded	✓	PASS
TC-06f	Message with embedded password fails	Contains credential pattern	Credential detected	✓	PASS
TC-06g	Valid CEO fraud message passes all checks	Valid message	0 errors	✓	PASS

1.3 Evaluator and Session Manager — Python

Files: TEST_unit_evaluator.py, TEST_unit_session_manager.py

Test ID	Test Class	Test Scenario	Expected	Result
TC-12	TestScoreToGrade	All 10 grade boundaries (0–100)	All correct	PASS (10/10)
TC-12	TestRuleBasedEvaluation	Phishing/scam/vishing keywords → attack_identified	Correct flags	PASS (5/5)
TC-12	TestRuleBasedEvaluation	Suspicion keywords → suspicion_shown	Correct flags	PASS (4/4)
TC-12	TestRuleBasedEvaluation	Compliance keywords → compliance_level=full	Correct flags	PASS (3/3)

TC-12	TestRuleBasedEvaluation	Neutral response → questioning	questioning	PASS (2/2)
TC-12	TestMergeEvaluations	Score blended 40% rule / 60% LLM	Correct blend	PASS (3/3)
TC-12	TestComputeSessionScore	Session aggregation — all scenarios	Correct	PASS (8/8)
TC-08	TestCreateSession	Session creation, retrieval, independence	All correct	PASS (6/6)
TC-13	TestAppendTurn	Turn logging, trigger accumulation, fault tolerance	All correct	PASS (4/4)
TC-13	TestGetConversationHistory	History build — attacker+user, null user excluded	All correct	PASS (3/3)
TC-08	TestEndSession	End session, duration, inactive check	All correct	PASS (4/4)
TC-06	TestStripReasoning	Reasoning stripping from model output	All correct	PASS (6/6)

Result: 58 / 58 passed in 0.36 s.

2. Integration Tests

File: BACKEND/TEST_integration_simulation.test.js

Test ID	Requirement	Test Scenario	Expected	Actual	Result
TC-07a	FR 3.3	User record stores profile fields after registration	Fields present in DB	✓	PASS
TC-07b	FR 3.3	User profile accessible via API when authenticated	200	✓	PASS

TC-07c	FR 3.3	Unauthenticated profile request rejected	401	401	PASS
TC-08a	FR 4.1	Active scenarios retrievable from API	200, array returned	✓	PASS
TC-08b	FR 4.1	Scenario document contains all required fields	All fields present	✓	PASS
TC-08c	FR 4.1	Start with unknown scenarioId returns 4xx	404, success:false	404	PASS
TC-09	FR 4.2	Simulation start responds within 10 s	< 10000 ms	182 ms	PASS
TC-10a	FR 4.4	Session records scenario type and persona	Fields in MongoDB	✓	PASS
TC-10b	FR 4.4	Turn records RAG-sourced triggers	Triggers array populated	✓	PASS
TC-11a	FR 4.5	"phishing" keyword → attack identified	attack_identified=true	✓	PASS
TC-11b	FR 4.5	"suspicious, verify" → suspicion shown	suspicion_shown=true	✓	PASS
TC-11c	FR 4.5	"sure, I'll do that" → complied	compliance_level=full	✓	PASS
TC-11d	FR 4.5	Neutral response → questioning	user_reaction_hint=questioning	✓	PASS
TC-11e	FR 4.5	Suspicion flag persisted to MongoDB	suspicionShown=true in DB	✓	PASS
TC-13a	FR 7.1	Completed session has all KPI fields	score, grade, status, duration	✓	PASS

TC-13b	FR 7.1	Session createdAt timestamp set	Date present	✓	PASS
TC-13c	FR 7.1	History endpoint returns completed sessions	Array returned	✓	PASS

Result: 18 / 18 passed.

3. System / End-to-End Tests

File: BACKEND/TEST_e2e_flows.test.js

Test ID	Requirement	Test Scenario	Expected	Actual	Result
TC-14a	FR 2.1	Valid registration creates user in DB	User found in DB	✓	PASS
TC-14b	FR 2.1	Duplicate email rejected	4xx	✓	PASS
TC-14c	FR 2.1	Password stored as bcrypt hash	Starts with \$2, compare passes	✓	PASS
TC-15a	FR 3.1	Scenario preview returns persona and goal	Data returned	✓	PASS
TC-15b	FR 3.1	Unauthenticated access to scenario blocked	401/403/404	✓	PASS
TC-15c	FR 3.1	Scenario has attackerPersona and description	Both fields present	✓	PASS
TC-16a	FR 2.2	Authenticated user accesses simulation history	200	✓	PASS
TC-16b	FR 2.2	Completed session appears in history	Data found	✓	PASS

TC-17a	FR 6.1	Completed session contains all KPI fields	All present	✓	PASS
TC-17b	FR 6.1	Susceptibility score in range 0–100	$0 \leq \text{score} \leq 100$	✓	PASS
TC-18a	FR 4.1	Scenario list includes required fields	scenarioId, type, difficulty	✓	PASS
TC-18b	FR 4.1	Inactive scenario returns 4xx on start	404	404	PASS
TC-19a	FR 2.4	Forgot-password sets resetPasswordToken	Token set in DB	✓	PASS
TC-19b	FR 2.4	Invalid reset token rejected	4xx	✓	PASS
TC-20a	FR 7.1	Only completed sessions in history	All status=completed	✓	PASS
TC-20b	FR 7.1	Session detail returns full turn data	Turns array with fields	✓	PASS

Result: 16 / 16 passed.

4. Black Box Functional Tests

File: BACKEND/TEST_blackbox_functional.test.js

Test ID	Test Scenario	Input	Expected	Actual	Result
BB-01a	Scenario start — missing scenarioId	{}	4xx, success:false	✓	PASS
BB-01b	Scenario start — null scenarioId	{scenarioId: null}	4xx	✓	PASS
BB-01c	SQL injection input	'; DROP TABLE	No 500 crash	✓	PASS

BB-02	Scenario list exposes attackerPersona	GET /scenarios	Persona field present	✓	PASS
BB-03a	Trainee blocked from admin route	GET /admin/users	401/403/404	✓	PASS
BB-03b	Unauthenticated blocked from protected route	No cookie	401	401	PASS
BB-03c	Admin can access admin route	Admin cookie	200	✓	PASS
BB-03d	Trainee blocked from analytics	GET /analytics	401/403	✓	PASS
BB-04a	Valid CEO message passes format checks	CEO message	0 errors	✓	PASS
BB-04b	Valid IT Support message passes format checks	IT message	0 errors	✓	PASS
BB-04c	Valid escalation message passes format checks	Escalation message	0 errors	✓	PASS
BB-04d	Empty message fails format check	""	Error detected	✓	PASS
BB-04e	Message over 500 characters fails	501-char string	Error detected	✓	PASS
BB-04f	Message with URL fails	Contains https://	Error detected	✓	PASS

BB-05	Manual abort → session status = completed	End session	status=completed in DB	✓	PASS
BB-06	Debrief shows overallScore and grade	GET session detail	Both fields present	✓	PASS
BB-07a	Single-character name — no crash	name = "A"	200–500, no crash	✓	PASS
BB-07b	Case-insensitive duplicate email rejected	CASE@x.com then case@x.com	Second rejected	✓	PASS
BB-07c	Missing name field rejected	No name	4xx	✓	PASS
BB-08a	Expired JWT rejected	-1 s token	401	401	PASS
BB-08b	Tampered JWT rejected	fake.token	401	401	PASS
BB-08c	No token → 401	No cookie	401	401	PASS

Result: 22 / 22 passed.

5. Non-Functional Tests

File: BACKEND/TEST_nonfunctional.test.js

Test ID	NFR	Test Scenario	Threshold	Measured	Result
NFR-01a	Performance	POST /auth/register response time	< 2000 ms	~880 ms	PASS
NFR-01b	Performance	POST /auth/login response time	< 2000 ms	~1200 ms	PASS

NFR-01c	Performance	GET /scenarios response time	< 2000 ms	~1138 ms	PASS
NFR-01d	Performance	GET /simulation/history response time	< 2000 ms	~1118 ms	PASS
NFR-02	LLM Performance	AI service health check response	< 500 ms	Skipped (not running)*	PASS
NFR-03a	Availability	Core endpoints return structured response	Not 500	✓	PASS
NFR-03b	Availability	5 repeated calls — no degradation	Not 500	✓	PASS
NFR-04a	Scalability	10 concurrent logins — no crash	All respond	✓	PASS
NFR-04b	Scalability	10 concurrent registrations — no crash	All respond	✓	PASS
NFR-05a	Security	Protected route without token → 401	401	401	PASS
NFR-05b	Security	HTTP security headers present	Headers exist	✓	PASS
NFR-05c	Security	CORS restricts to configured origins	No wildcard *	✓	PASS
NFR-06a	Security	Login does not expose password in body	Password excluded	✓	PASS
NFR-06b	Security	/user/me does not return password field	password = undefined	✓	PASS
NFR-07a	Usability	All endpoints return JSON content-type	application/json	✓	PASS
NFR-07b	Usability	Error responses have success:false + message	Structured	✓	PASS
NFR-09a	Reliability	Malformed JSON returns 4xx not 500	4xx	✓	PASS

NFR-09b	Reliability	Oversized payload handled gracefully	Not 500	✓	PASS
NFR-09c	Reliability	Unexpected fields ignored safely	Not 500	✓	PASS
NFR-12	Monitoring	Session doc has all monitoring fields	All present	✓	PASS

AI service was not running during NFR-02 execution - test skips gracefully by design.

Result: 19 / 19 passed.

6. AI Service Integration Tests - Python

File: BACKEND/ai_service/TEST_integration_api.py

Test ID	Test Class	Test Scenario	Expected	Result
NFR-03	TestHealthEndpoint	Health endpoint returns 200	200	PASS
NFR-03	TestHealthEndpoint	Health response body has status=ok	"ok"	PASS
NFR-01	TestHealthEndpoint	Health responds within 500 ms	< 500 ms	PASS
TC-08	TestSimulationStart	Start returns session_id	session_id present	PASS
TC-08	TestSimulationStart	Attacker message under 500 characters	len < 500	PASS
TC-08	TestSimulationStart	Turn number = 1	turn_number = 1	PASS
TC-08	TestSimulationStart	triggers_fired is a list	list type	PASS
TC-08	TestSimulationStart	response_options = 4 options (A/B/C/D)	4 options	PASS
TC-08	TestSimulationStart	Start returns 200	200	ERROR*

TC-08	TestSimulationStart	Attacker message is non-empty string	len > 0	ERROR*
TC-09	TestSimulationRespond	Stream returns done event	done event present	PASS
TC-09	TestSimulationRespond	Done event has attacker_message	field present	PASS
TC-09	TestSimulationRespond	Done event has evaluation	field present	PASS
TC-09	TestSimulationRespond	response_options in done event	field present	PASS
TC-09	TestSimulationRespond	Eval event precedes tokens in stream	ordering correct	PASS
TC-09	TestSimulationRespond	Invalid session returns 404	404	PASS
TC-10	TestAdaptiveTacticRetrieval	triggers_fired list returned	list present	PASS
TC-10	TestAdaptiveTacticRetrieval	Triggers are string values	string type	ERROR*
TC-10	TestAdaptiveTacticRetrieval	Attacker message is contextually relevant	relevant content	PASS
TC-11	TestSuspicionDetection	Compliant response detected as complied	complied	PASS
TC-11	TestSuspicionDetection	Refusal response detected as refused	refused	SKIP**
TC-11	TestSuspicionDetection	Attack identified response flagged	attack_identified	SKIP**
TC-13	TestSimulationEnd	End returns summary	summary present	SKIP**
TC-13	TestSimulationEnd	Summary has overall_score and grade	both present	SKIP**
TC-13	TestSimulationEnd	Invalid session returns 404	404	PASS

httpx.ReadTimeout - /simulate/start exceeded 30 s due to OpenRouter free-tier rate limiting (50 req/day). Not a code defect.

Session fixture consumed by prior tests in the same run.

Result: 19 passed, 4 skipped, 3 rate-limit timeouts. Total runtime: 16 minutes 11 seconds.

Test Execution Output

Node.js - Jest (6 suites)

Test Suites: 6 passed, 6 total

Tests: 120 passed, 120 total

Time: 17.541 s

Python - Unit Tests

58 passed in 0.36s

Python - Integration Tests

19 passed, 4 skipped, 3 errors in 971.51s (0:16:11)

Critical Evaluation:

1. Deviations from Initial Design

The initial report proposed a specific technical stack that evolved considerably during development. These were not failures of planning - they were informed decisions made when practical constraints made the original choices unsuitable.

1.1 LangChain Dropped - Custom Pipeline Built Instead

The initial design specified LangChain as the orchestration layer, responsible for managing the agent loop, coordinating retrieval, and chaining LLM calls. In practice, LangChain was abandoned early in development for three reasons.

First, LangChain's abstractions added complexity without benefit for this use case. The project does not need a general-purpose agent that decides which tools to call - it needs a fixed, predictable pipeline: retrieve tactics, build prompt, generate message, evaluate response, log results.

LangChain's agent framework is designed for open-ended reasoning tasks; forcing a structured simulation loop through it produced difficult-to-debug behaviour where the agent would occasionally skip steps or reformulate the retrieval query in unexpected ways.

Second, LangChain's Python library had version conflicts with the Python 3.14 environment used in development. Dependency resolution errors consumed significant time and the fixes introduced further instability.

Third, and most importantly, building the pipeline manually produced a system that is easier to understand, debug, and extend. Every step in the retrieval and generation process is explicit code - `retrieve_initial_attack()`, `generate_attacker_message()`, `evaluate_response()`, `compute_session_score()` - rather than hidden inside a framework's internal state machine. This transparency was particularly valuable during testing and debugging.

The custom pipeline built with FastAPI, httpx, and sentence-transformers delivers the same functionality that LangChain would have provided, with full visibility into every step.

1.2 Gemma Replaced with Gemma 4B and OpenRouter

The initial report specified Gemma as the language model, to be run locally via Ollama. During development, Gemma was tested locally and produced inconsistent output - reasoning tokens from the model's chain-of-thought process leaked into the generated attacker messages, requiring additional stripping logic that still did not fully resolve the issue. The model also showed higher hallucination rates in JSON output tasks, which caused frequent parse failures in the evaluation pipeline.

Gemma 4B, also run through Ollama, produced cleaner outputs with less reasoning leakage, better adherence to prompt constraints, and more consistent JSON formatting in evaluation responses. The switch was straightforward since both models are served through the same Ollama API.

OpenRouter was introduced as a cloud-based alternative that allows the system to use more capable models without the hardware requirements of local inference. The current configuration supports both Ollama (local) and OpenRouter (cloud) through a single environment variable switch, giving the system flexibility depending on the deployment environment. The primary limitation encountered was OpenRouter's free-tier rate limit of 50 requests per day, which caused timeout errors in the AI service integration test suite.

1.3 Next.js Server Actions Replaced with REST API Calls

The initial architecture described Next.js server actions as the API layer between the frontend and backend services. During implementation this was replaced with a standard REST approach - the Next.js frontend makes fetch calls to the Express backend, and the Express backend handles all business logic and database access.

The reason was separation of concerns. Server actions blur the boundary between frontend and backend code, making it harder to test, document, and scale each layer independently. A dedicated Express backend gives the admin panel, the frontend, and any future mobile client a single consistent API to call. It also allows the Express backend to be the sole entry point to the AI service, ensuring the AI layer is never publicly exposed.

1.4 MongoDB Schema Evolved from Initial ER Diagram

The initial ER diagram specified a TargetProfile entity as a separate collection linked one-to-one to the User. In the implemented system, the profile data - job role, department, technical level, and vulnerability tags - is stored directly on the User document as fields and a profileTags array. This simplification eliminates an unnecessary join and reduces query complexity, since the profile data is always needed alongside the user record.

The Assessment entity proposed in the initial design was also simplified. Rather than a separate collection of scored OCEAN personality dimensions, the onboarding questionnaire feeds directly into the profileTags array on the user document. This provides the same personalisation capability - the AI service uses the tags to shape attacks - without the overhead of a separate assessment pipeline.

2. What Worked Well

2.1 The RAG Pipeline

The two-stage retrieval pipeline - BGE embedding model for approximate nearest neighbour search in Pinecone, followed by cross-encoder reranking - worked exactly as intended. The attacker messages are noticeably more specific and realistic compared to prompting the LLM without retrieved context. The metadata filtering by scenario_type and chunk_type ensures that phishing scenarios draw from phishing knowledge and debrief queries draw from debrief content, preventing irrelevant tactics from contaminating the prompt.

The decision to use a cross-encoder reranker rather than relying solely on vector similarity was validated during testing - without reranking, some retrieved chunks were semantically adjacent but contextually inappropriate for the attack stage.

2.2 SSE Streaming Architecture:

The Server-Sent Events streaming pipeline - from Ollama through FastAPI through Express to the browser - produced the intended typewriter effect and made the simulation feel live and immediate. The decision to use SSE over WebSockets was correct for this use case: the communication is one-directional during streaming (server to client), uses standard HTTP connections, and requires no handshake or persistent connection management. The parallel evaluation thread - running LLM evaluation concurrently while the attacker message streams - kept the turn-around time low without blocking the user experience.

2.3 Authentication System

The layered authentication system - bcrypt hashing at cost factor 12, JWT in HttpOnly cookies, SHA-256 hashed reset tokens, TOTP-based two-factor authentication, rate limiting on auth endpoints, and role-based access control - covers the major threat vectors comprehensively. The decision to store only token hashes in the database, never raw tokens, means a database read compromise cannot be used to verify emails or reset passwords. The 120 JavaScript test suite results confirmed all authentication behaviours work correctly at every boundary.

2.4 Modular Four-Service Architecture

Running the frontend, admin panel, Express backend, and AI service as independent processes proved to be the right architectural decision. During development, the AI service could be restarted for model changes without touching the Express backend or frontend. The backend could be updated without restarting the Next.js development server. This independence also made testing easier - the JavaScript test suites run against the backend with an in-memory MongoDB instance and no dependency on the AI service being live.

3. Limitations and What Did Not Work as Planned

3.1 OpenRouter Free-Tier Rate Limiting

The most significant operational constraint encountered was OpenRouter's free-tier rate limit of 50 requests per day. This is not a system design problem but it did affect the AI service integration test suite, which produced 3 timeout errors due to the limit being hit mid-run. In a production deployment with a paid API tier this constraint would not exist. The system is built to switch between local Ollama and OpenRouter via a single environment variable, so upgrading to a paid tier requires no code changes.

3.2 In-Memory AI Sessions

AI session state is held in a Python dictionary in the FastAPI process memory. This means if the AI service crashes or is restarted during an active simulation, the session is lost and the user cannot continue. The Express backend stores all turn data in MongoDB so the record is not lost, but the simulation cannot be resumed. A more robust implementation would persist session state to Redis or MongoDB with a short TTL, allowing sessions to survive service restarts.

3.3 Vishing and Deepfake Scenarios Not Implemented

The initial case studies highlighted vishing (voice phishing) and AI deepfake video attacks as the most damaging real-world vectors. The current system is text-only. Implementing a realistic vishing simulation would require voice synthesis and speech recognition integration, which was outside the scope of this project phase. The pretexting and MFA fatigue scenarios approximate these attack types in text form, but a voice-based channel would significantly increase training realism.

3.4 Pinecone Free Tier Constraints

Pinecone's free tier limits index size and query throughput. The current knowledge base is relatively small - five scenario files and one psychological triggers file - so this has not caused issues. Scaling the knowledge base with more granular tactics, more scenario variants, and multilingual content would require a paid Pinecone tier or migration to a self-hosted vector database such as Qdrant.

4. Tool Evaluation

Tool	Assessment
Next.js 14	Effective for both frontend and admin panel. App Router provides clean route organisation. Tailwind CSS kept styling consistent without a component library dependency.
Express.js	Straightforward and well-suited. Middleware chain is easy to reason about and test. Would consider Fastify for a production rebuild due to better performance characteristics.
FastAPI	Excellent choice for the AI service. Native async support, automatic OpenAPI documentation, and Pydantic validation reduced boilerplate significantly.
MongoDB / Mongoose	Appropriate for the document-oriented data model. The flexible schema allowed the User model to evolve during development without migrations.
Pinecone	Reliable and fast for vector search. Free tier is limiting for scale but adequate for the current knowledge base.
Gemma 7B via Ollama	Good output quality for a 7B parameter model. Runs locally without API costs. Reasoning leakage required stripping logic but was manageable.
OpenRouter	Useful for accessing larger models without local hardware. Free-tier rate limits make it unsuitable for sustained testing; a paid tier is required for production use.
Jest	Excellent JavaScript test runner. The in-memory MongoDB setup via mongodb-memory-server made integration tests fast and fully isolated.

pytest	Clean and readable for Python unit tests. The 58 unit tests ran in 0.36 seconds, demonstrating good isolation.
bcrypt / JWT / speakeasy	Robust and well-maintained libraries. No issues encountered. The combination of HttpOnly cookies, bcrypt, and TOTP provides strong authentication security.

5. Future Work

Given additional time and resources, the following improvements would be prioritized:

- Voice simulation for vishing. Integrating a text-to-speech engine for attacker messages and speech-to-text for user responses would allow the vishing scenario to operate as a realistic phone call simulation - the highest-impact social engineering vector currently approximated only in text.
- Persistent AI sessions. Moving session state from in-memory Python dictionaries to Redis with a short TTL would make the simulation fault-tolerant across service restarts and enable horizontal scaling of the AI service.
- Expanded knowledge base. The current knowledge base covers five scenario types at a high level. A production system would benefit from granular, industry-specific tactic libraries - different knowledge for finance teams versus IT teams versus HR, reflecting the way real attackers tailor their approaches.
- Organizational multi-tenancy. The current system supports individual users and a single admin view. A full enterprise deployment would require tenant isolation, so that one organization's simulation data is never visible to another, with per-organization KPI dashboards and department-level analytics.
- Real email and SMS channel integration. Conducting simulated phishing campaigns via actual email and SMS - rather than through the chat interface - would create higher-fidelity training scenarios and allow organizations to test employees in their real working environment rather than a dedicated training platform.

Conclusion:

INTERCEPTOR set out to address a genuine gap in cybersecurity awareness training - the inability of traditional static training to replicate the adaptive, conversational pressure of a real social engineering attack. The platform delivers on that goal. Users engage in live text-based simulations against an AI attacker whose messages are grounded in a retrieved knowledge base of real social engineering tactics, adapt to how the user responds, and are personalised to the user's job role and assessed vulnerability profile. After each session, users receive a scored debrief that identifies exactly which psychological triggers were used against them and provides specific counter-measures - feedback that is directly tied to their individual behaviour rather than generic advice.

The development process required several significant departures from the initial design. LangChain was replaced with a custom orchestration pipeline that is more transparent and maintainable. Llama 3.1 was replaced with Gemma 7B after output quality issues, with OpenRouter added as a cloud inference alternative. Next.js server actions were replaced with a dedicated Express REST API that provides a clean, testable boundary between the frontend and backend layers. Each deviation was driven by practical constraints encountered during implementation and resulted in a more stable and understandable system than the original specification would have produced.

The final system comprises four independently running services, 204 automated tests achieving a 97% pass rate, a complete authentication system with two-factor authentication support, a working RAG pipeline with two-stage retrieval and reranking, real-time streaming simulation, a scoring and debrief engine, structured learning modules, a phishing lab, and a full administrative dashboard. The limitations that remain - OpenRouter rate constraints, in-memory session state, and the absence of voice-based phishing simulation - are well understood and addressable in future development phases. INTERCEPTOR demonstrates that AI-driven adaptive simulation is a viable and effective approach to social engineering awareness training, and provides a solid foundation for a production-ready platform.

References:

- [1] Heartfield, R. and Loukas, G. (2015) 'A Taxonomy of Attacks and a Survey of Defence Mechanisms for Semantic Social Engineering Attacks', *ACM Computing Surveys*, 48(3), pp. 1–39. Available at: <https://dl.acm.org/doi/10.1145/2835375>
- [2] Kaur, G. et al. (2021) 'Social Engineering in Cybersecurity: Effect Mechanisms, Human Vulnerabilities and Attack Methods', *IEEE Access*, 9, pp. 11895–11923. Available at: <https://ieeexplore.ieee.org/document/9323026/>
- [3] Alotaibi, F. et al. (2024) 'A Comprehensive Survey on Social Engineering Attacks, Countermeasures, Case Study, and Research Challenges', *Information Processing and Management*, 61(4). Available at: <https://dl.acm.org/doi/10.1016/j.ipm.2024.103928>
- [4] Alkhalil, Z. et al. (2024) 'A Comprehensive Analysis of Social Engineering Attacks: From Phishing to Prevention', *IEEE Conference Publication*. Available at: <https://ieeexplore.ieee.org/document/10696444/>
- [5] Ferreira, A. et al. (2022) 'A Study on the Psychology of Social Engineering-Based Cyberattacks and Existing Countermeasures', *Applied Sciences*, 12(12), p. 6042. Available at: <https://www.mdpi.com/2076-3417/12/12/6042>
- [6] Gao, Y. et al. (2024) 'Retrieval-Augmented Generation for Large Language Models: A Survey', *arXiv preprint arXiv:2312.10997*. Available at: <https://arxiv.org/abs/2312.10997>
- [7] Fan, W. et al. (2024) 'A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models', *arXiv preprint arXiv:2405.06211*. Available at: <https://arxiv.org/pdf/2405.06211>
- [8] Gupta, M. et al. (2024) 'Large Language Models for Cyber Security: A Systematic Literature Review', *arXiv preprint arXiv:2405.04760*. Available at: <https://arxiv.org/html/2405.04760v5>
- [9] Yao, J. et al. (2024) 'Machine Learning and Neural Networks for Phishing Detection: A Systematic Review (2017–2024)', *Electronics*, 14(18). Available at: <https://www.mdpi.com/2079-9292/14/18/3744>
- [10] Vishwanath, A. et al. (2024) 'Assessing the Effect of Cybersecurity Training on End-users: A Meta-analysis', *Computers and Security*, ScienceDirect. Available at: <https://www.sciencedirect.com/science/article/pii/S016740482400511X>
- [11] Fortinet (2024) *2024 Security Awareness and Training Global Research Report*. Available at: <https://www.fortinet.com/content/dam/fortinet/assets/reports/report-2024-security-awareness-and-training.pdf>

- [12] Hadnagy, C. (2018) *Social Engineering: The Science of Human Hacking*. 2nd edn. Indianapolis: Wiley.
- [13] InfoQ (2022) 'Multi-Factor Authentication Fatigue Key Factor in Uber Breach', *InfoQ*, September. Available at: <https://www.infoq.com/news/2022/09/Uber-breach-mfa-fatigue/>
- [14] Morphisec (2023) 'Threat Analysis: MGM Resorts International ALPHV/Blackcat/Scattered Spider Ransomware Attack'. Available at: <https://www.morphisec.com/blog/mgm-resorts-alphv-spider-ransomware-attack/>
- [15] CNN Business (2024) 'Arup revealed as victim of \$25 million deepfake scam involving Hong Kong employee', *CNN*, May. Available at: <https://www.cnn.com/2024/05/16/tech/arup-deepfake-scam-loss-hong-kong-intl-hnk>
- [16] World Economic Forum (2025) 'Cybercrime: Lessons learned from a \$25m deepfake attack'. Available at: <https://www.weforum.org/stories/2025/02/deepfake-ai-cybercrime-arup/>
- [17] Springer Nature (2025) 'The Rise of Hallucination in Large Language Models: Systematic Reviews, Performance Analysis and Challenges', *Cluster Computing*. Available at: <https://link.springer.com/article/10.1007/s10586-025-05891-z>
- [18] Palo Alto Networks (2024) 'AI, Cybersecurity and the Rise of Large Language Models'. Available at: <https://www.paloaltonetworks.com/blog/2024/04/ai-cybersecurity-and-large-language-models/>